



RTL Design Sherpa

RTL AMBA AXI5-Lite Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXI5-Lite (AXIL5) Modules.....	16
1.1 Overview.....	16
1.2 Functional Description.....	16
1.3 Usage Example.....	17
1.4 Design Notes.....	18
1.5 Related Modules.....	18
1.5.1 Transport (4).....	18
1.5.2 Clock-gated (4).....	18
1.5.3 Monitored (4).....	19
1.5.4 Monitored and clock-gated (4).....	19
1.6 Testing.....	19
1.7 Navigation.....	20
2 AXIL5 Master Read.....	20
2.1 Overview.....	20
2.2 Parameters.....	20
2.3 Ports.....	21
2.4 Functional Description.....	23
2.4.1 Optional signal groups.....	23
2.4.2 With every group disabled, this IS the AXI4-Lite module.....	24
2.4.3 It transports; it does not interpret.....	24
2.5 Timing Characteristics.....	25
2.5.1 Buffer Depths and Latency.....	25

2.5.2 Optional-group effect.....	25
2.6 Usage Examples.....	25
2.7 Design Notes.....	26
2.8 Related Modules.....	27
2.9 Testing.....	27
2.10 Navigation.....	27
3 AXIL5 Master Read, Clock-Gated.....	27
3.1 Overview.....	27
3.2 Parameters.....	28
3.3 Ports.....	29
3.4 Functional Description.....	31
3.4.1 Optional signal groups.....	31
3.4.2 With every group disabled, this IS the AXI4-Lite module.....	32
3.4.3 It transports; it does not interpret.....	32
3.5 Timing Characteristics.....	32
3.5.1 Buffer Depths and Latency.....	32
3.5.2 Optional-group effect.....	33
3.6 Usage Examples.....	33
3.7 Design Notes.....	34
3.8 Related Modules.....	35
3.9 Testing.....	35
3.10 Navigation.....	35
4 AXIL5 Master Read Monitor.....	35
4.1 Overview.....	36

4.2 Parameters.....	36
4.3 Ports.....	40
4.4 Functional Description.....	45
4.4.1 Optional signal groups.....	45
4.4.2 With every group disabled, this IS the AXI4-Lite module.....	46
4.4.3 It transports; it does not interpret.....	46
4.5 Design Notes.....	47
4.6 Timing Characteristics.....	47
4.6.1 Buffer Depths and Latency.....	47
4.6.2 Optional-group effect.....	48
4.7 Usage Examples.....	48
4.8 Related Modules.....	49
4.9 Testing.....	50
4.10 Navigation.....	50
5 AXI5 Master Read Monitor, Clock-Gated.....	50
5.1 Overview.....	50
5.2 Parameters.....	51
5.3 Ports.....	53
5.4 Functional Description.....	59
5.4.1 Optional signal groups.....	59
5.4.2 With every group disabled, this IS the AXI4-Lite module.....	59
5.4.3 It transports; it does not interpret.....	60
5.5 Design Notes.....	60
5.6 Timing Characteristics.....	61

5.6.1 Buffer Depths and Latency.....	61
5.6.2 Optional-group effect.....	61
5.7 Usage Examples.....	61
5.8 Related Modules.....	63
5.9 Testing.....	63
5.10 Navigation.....	63
6 AXI5-Lite Master Read with Lite Monitor.....	63
6.1 Overview.....	63
6.1.1 What is not here.....	64
6.1.2 Key Features.....	64
6.2 Parameters.....	64
6.2.1 Monitor Parameters.....	64
6.2.2 Core Parameters (passed through to axil5_master_rd).....	65
6.3 Ports.....	67
6.3.1 Monitor Control, Bus and Status.....	67
6.3.2 Core Ports (as on axil5_master_rd).....	68
6.4 Functional Description.....	70
6.5 Timing Characteristics.....	70
6.6 Usage Examples.....	71
6.7 Design Notes.....	71
6.8 Related Modules.....	72
6.9 Testing.....	72
6.10 Navigation.....	72
7 AXIL5 Master Write.....	72

7.1 Overview.....	72
7.2 Parameters.....	73
7.3 Ports.....	74
7.4 Functional Description.....	76
7.4.1 Optional signal groups.....	76
7.4.2 With every group disabled, this IS the AXI4-Lite module.....	77
7.4.3 It transports; it does not interpret.....	77
7.5 Timing Characteristics.....	78
7.5.1 Buffer Depths and Latency.....	78
7.5.2 Optional-group effect.....	78
7.6 Usage Examples.....	78
7.7 Design Notes.....	79
7.8 Related Modules.....	80
7.9 Testing.....	80
7.10 Navigation.....	80
8 AXI5 Master Write, Clock-Gated.....	81
8.1 Overview.....	81
8.2 Parameters.....	81
8.3 Ports.....	82
8.4 Functional Description.....	85
8.4.1 Optional signal groups.....	85
8.4.2 With every group disabled, this IS the AXI4-Lite module.....	85
8.4.3 It transports; it does not interpret.....	86
8.5 Timing Characteristics.....	86

8.5.1 Buffer Depths and Latency.....	86
8.5.2 Optional-group effect.....	86
8.6 Usage Examples.....	87
8.7 Design Notes.....	88
8.8 Related Modules.....	89
8.9 Testing.....	89
8.10 Navigation.....	89
9 AXIL5 Master Write Monitor.....	89
9.1 Overview.....	89
9.2 Parameters.....	90
9.3 Ports.....	94
9.4 Functional Description.....	100
9.4.1 Optional signal groups.....	100
9.4.2 With every group disabled, this IS the AXI4-Lite module.....	100
9.4.3 It transports; it does not interpret.....	101
9.5 Design Notes.....	101
9.6 Timing Characteristics.....	101
9.6.1 Buffer Depths and Latency.....	101
9.6.2 Optional-group effect.....	102
9.7 Usage Examples.....	102
9.8 Related Modules.....	104
9.9 Testing.....	104
9.10 Navigation.....	104
10 AXIL5 Master Write Monitor, Clock-Gated.....	104

10.1 Overview.....	104
10.2 Parameters.....	105
10.3 Ports.....	108
10.4 Functional Description.....	113
10.4.1 Optional signal groups.....	113
10.4.2 With every group disabled, this IS the AXI4-Lite module.....	114
10.4.3 It transports; it does not interpret.....	114
10.5 Design Notes.....	115
10.6 Timing Characteristics.....	115
10.6.1 Buffer Depths and Latency.....	115
10.6.2 Optional-group effect.....	116
10.7 Usage Examples.....	116
10.8 Related Modules.....	117
10.9 Testing.....	118
10.10 Navigation.....	118
11 AXI5-Lite Master Write with Lite Monitor.....	118
11.1 Overview.....	118
11.1.1 What is not here.....	118
11.1.2 Key Features.....	119
11.2 Parameters.....	119
11.2.1 Monitor Parameters.....	119
11.2.2 Core Parameters (passed through to axil5_master_wr).....	120
11.3 Ports.....	122
11.3.1 Monitor Control, Bus and Status.....	122

11.3.2 Core Ports (as on axil5_master_wr).....	123
11.4 Functional Description.....	125
11.5 Timing Characteristics.....	126
11.6 Usage Examples.....	126
11.7 Design Notes.....	127
11.8 Related Modules.....	127
11.9 Testing.....	127
11.10 Navigation.....	127
12 AXIL5 Slave Read.....	128
12.1 Overview.....	128
12.2 Parameters.....	128
12.3 Ports.....	129
12.4 Functional Description.....	131
12.4.1 Optional signal groups.....	131
12.4.2 With every group disabled, this IS the AXI4-Lite module.....	132
12.4.3 It transports; it does not interpret.....	132
12.5 Timing Characteristics.....	132
12.5.1 Buffer Depths and Latency.....	132
12.5.2 Optional-group effect.....	133
12.6 Usage Examples.....	133
12.7 Design Notes.....	134
12.8 Related Modules.....	134
12.9 Testing.....	135
12.10 Navigation.....	135

13 AXIL5 Slave Read, Clock-Gated.....	135
13.1 Overview.....	135
13.2 Parameters.....	136
13.3 Ports.....	137
13.4 Functional Description.....	139
13.4.1 Optional signal groups.....	139
13.4.2 With every group disabled, this IS the AXI4-Lite module.....	139
13.4.3 It transports; it does not interpret.....	140
13.5 Timing Characteristics.....	140
13.5.1 Buffer Depths and Latency.....	140
13.5.2 Optional-group effect.....	141
13.6 Usage Examples.....	141
13.7 Design Notes.....	142
13.8 Related Modules.....	143
13.9 Testing.....	143
13.10 Navigation.....	143
14 AXIL5 Slave Read Monitor.....	143
14.1 Overview.....	143
14.2 Parameters.....	144
14.3 Ports.....	148
14.4 Functional Description.....	153
14.4.1 Optional signal groups.....	153
14.4.2 With every group disabled, this IS the AXI4-Lite module.....	154
14.4.3 It transports; it does not interpret.....	154

14.5 Design Notes.....	155
14.6 Timing Characteristics.....	155
14.6.1 Buffer Depths and Latency.....	155
14.6.2 Optional-group effect.....	155
14.7 Usage Examples.....	156
14.8 Related Modules.....	157
14.9 Testing.....	157
14.10 Navigation.....	158
15 AXIL5 Slave Read Monitor, Clock-Gated.....	158
15.1 Overview.....	158
15.2 Parameters.....	158
15.3 Ports.....	161
15.4 Functional Description.....	167
15.4.1 Optional signal groups.....	167
15.4.2 With every group disabled, this IS the AXI4-Lite module.....	167
15.4.3 It transports; it does not interpret.....	168
15.5 Design Notes.....	168
15.6 Timing Characteristics.....	168
15.6.1 Buffer Depths and Latency.....	168
15.6.2 Optional-group effect.....	169
15.7 Usage Examples.....	169
15.8 Related Modules.....	170
15.9 Testing.....	171
15.10 Navigation.....	171

16 AXI5-Lite Slave Read with Lite Monitor.....	171
16.1 Overview.....	171
16.1.1 What is not here.....	172
16.1.2 Key Features.....	172
16.2 Parameters.....	172
16.2.1 Monitor Parameters.....	172
16.2.2 Core Parameters (passed through to axil5_slave_rd).....	173
16.3 Ports.....	175
16.3.1 Monitor Control, Bus and Status.....	175
16.3.2 Core Ports (as on axil5_slave_rd).....	176
16.4 Functional Description.....	178
16.5 Timing Characteristics.....	178
16.6 Usage Examples.....	178
16.7 Design Notes.....	179
16.8 Related Modules.....	180
16.9 Testing.....	180
16.10 Navigation.....	180
17 AXIL5 Slave Write.....	180
17.1 Overview.....	180
17.2 Parameters.....	181
17.3 Ports.....	182
17.4 Functional Description.....	184
17.4.1 Optional signal groups.....	184
17.4.2 With every group disabled, this IS the AXI4-Lite module.....	185

17.4.3 It transports; it does not interpret.....	185
17.5 Timing Characteristics.....	185
17.5.1 Buffer Depths and Latency.....	185
17.5.2 Optional-group effect.....	186
17.6 Usage Examples.....	186
17.7 Design Notes.....	187
17.8 Related Modules.....	187
17.9 Testing.....	188
17.10 Navigation.....	188
18 AXIL5 Slave Write, Clock-Gated.....	188
18.1 Overview.....	188
18.2 Parameters.....	189
18.3 Ports.....	190
18.4 Functional Description.....	192
18.4.1 Optional signal groups.....	192
18.4.2 With every group disabled, this IS the AXI4-Lite module.....	193
18.4.3 It transports; it does not interpret.....	193
18.5 Timing Characteristics.....	194
18.5.1 Buffer Depths and Latency.....	194
18.5.2 Optional-group effect.....	194
18.6 Usage Examples.....	194
18.7 Design Notes.....	195
18.8 Related Modules.....	196
18.9 Testing.....	197

18.10 Navigation.....	197
19 AXIL5 Slave Write Monitor.....	197
19.1 Overview.....	197
19.1.1 Key Features.....	197
19.2 Parameters.....	198
19.3 Ports.....	202
19.4 Functional Description.....	207
19.4.1 With every group disabled, this IS the AXI4-Lite module.....	208
19.4.2 It transports; it does not interpret.....	208
19.5 Design Notes.....	209
19.6 Timing Characteristics.....	209
19.6.1 Buffer Depths and Latency.....	209
19.6.2 Optional-group effect.....	210
19.7 Usage Examples.....	210
19.8 Related Modules.....	211
19.9 Testing.....	212
19.10 Navigation.....	212
20 AXIL5 Slave Write Monitor, Clock-Gated.....	212
20.1 Overview.....	212
20.1.1 Key Features.....	212
20.2 Parameters.....	213
20.3 Ports.....	215
20.4 Functional Description.....	221
20.4.1 With every group disabled, this IS the AXI4-Lite module.....	222

20.4.2 It transports; it does not interpret.....	222
20.5 Design Notes.....	222
20.6 Timing Characteristics.....	223
20.6.1 Buffer Depths and Latency.....	223
20.6.2 Optional-group effect.....	223
20.7 Usage Examples.....	224
20.8 Related Modules.....	225
20.9 Testing.....	225
20.10 Navigation.....	226
21 AXI5-Lite Slave Write with Lite Monitor.....	226
21.1 Overview.....	226
21.1.1 What is not here.....	226
21.1.2 Key Features.....	226
21.2 Parameters.....	227
21.2.1 Monitor Parameters.....	227
21.2.2 Core Parameters (passed through to axil5_slave_wr).....	228
21.3 Ports.....	229
21.3.1 Monitor Control, Bus and Status.....	229
21.3.2 Core Ports (as on axil5_slave_wr).....	231
21.4 Functional Description.....	233
21.5 Timing Characteristics.....	233
21.6 Usage Examples.....	234
21.7 Design Notes.....	234
21.8 Related Modules.....	235

21.9 Testing.....	235
21.10 Navigation.....	235

1 AXI5-Lite (AXIL5) Modules

Location: rtl/amba/axil5/ **Sibling family:** [AXI4-Lite](#)

1.1 Overview

Sixteen modules mirroring the AXI4-Lite family one for one, with the AXI5-Lite optional signal groups threaded through. Same channel set, same handshakes, same single-beat semantics – what AXI5-Lite adds is sideband.

If you came here looking for a full AXI5-Lite protocol stack, read the next section before you instantiate anything.

1.2 Functional Description

These modules **transport** AXI5-Lite signals. They do not execute AXI5-Lite semantics:

- MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. Nothing here interprets a partition ID or an encryption context.
- POISON is carried. It is never generated and never checked.
- LOCK is carried with no exclusive-access monitor behind it. A completer returning EXOKAY is not validated against anything.

Those behaviours belong to the endpoints on either side. Read this before treating the family as a full AXI5-Lite protocol stack – it is not one.

AXI4-Lite vs AXI5-Lite, as this library implements them:

Feature	AXI4-Lite	AXI5-Lite
Channels	AW, W, B, AR, R	same
Burst support	none	none
Transaction ID	none	none
User sideband	not present	AxUSER/WUSER/BUSER/

Feature	AXI4-Lite	AXI5-Lite
Trace	not present	RUSER, gated by ENABLE_USER AxTRACE/BTRACE/RTRACE, gated by ENABLE_TRACE
Loopback ID	not present	AxLOOP/BLLOOP/RLLOOP, gated by ENABLE_LOOP
MPAM / MECID / NSAID	not present	on the address channels, each independently gated
Poison	not present	WPOISON/RPOISON, one bit per 64 data bits
Exclusive access	not present	AxLOCK, gated by ENABLE_LOCK

With every group disabled the two are the same interface. The packed payload widths match channel for channel, which is what lets one testbench bind to either family.

1.3 Usage Example

Two configurations of the same module – every group off, and the security qualifiers on:

```
// AXI4-Lite-equivalent: every group off, payload widths identical to axil4
axil5_master_rd #(
    .AXIL_ADDR_WIDTH (32),
    .AXIL_DATA_WIDTH (32),
    .ENABLE_USER (0), .ENABLE_TRACE (0), .ENABLE_LOOP
(0), .ENABLE_MPAM (0),
    .ENABLE_MECID(0), .ENABLE_NSAID(0), .ENABLE_POISON(0), .ENABLE_LOCK (0)
) u_plain ( /* ... */ );

// With the security qualifiers, which is what AXI5-Lite is usually for
axil5_master_rd #(
    .AXIL_ADDR_WIDTH (32),
    .AXIL_DATA_WIDTH (32),
    .NSAID_WIDTH (4), .MPAM_WIDTH (11), .MECID_WIDTH (16),
    .ENABLE_NSAID(1), .ENABLE_MPAM(1), .ENABLE_MECID(1),
    .ENABLE_USER (0), .ENABLE_TRACE(0), .ENABLE_LOOP (0),

```

```

        .ENABLE_POISON(0), .ENABLE_LOCK(0)
    ) u_secure ( /* ... */ );

```

1.4 Design Notes

Known limitations:

- No protocol checking of the optional groups. The monitored variants observe handshakes, addresses, responses and timing; axi_monitor_filtered has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON.
 - No exclusive-access monitor. AxLOCK is transported; nothing tracks exclusive state or validates EXOKAY.
 - No poison generation or checking.
 - Timing diagrams are not yet drawn for this family.
-

1.5 Related Modules

The family, grouped by what each variant adds:

1.5.1 Transport (4)

Module	Channels
axil5_master_rd	AR, R
axil5_master_wr	AW, W, B
axil5_slave_rd	AR, R
axil5_slave_wr	AW, W, B

1.5.2 Clock-gated (4)

Module	Adds
axil5_master_rd_cg	One amba_clock_gate_ctrl over the whole inner module
axil5_master_wr_cg	One amba_clock_gate_ctrl over the whole inner module
axil5_slave_rd_cg	One amba_clock_gate_ctrl over the whole inner module

Module	Adds
axil5_slave_wr_cg	One amba_clock_gate_ctrl over the whole inner module

1.5.3 Monitored (4)

Module	Adds
axil5_master_rd_mon	axi_monitor_filtered, monbus packet output
axil5_master_wr_mon	axi_monitor_filtered, monbus packet output
axil5_slave_rd_mon	axi_monitor_filtered, monbus packet output
axil5_slave_wr_mon	axi_monitor_filtered, monbus packet output

1.5.4 Monitored and clock-gated (4)

Module	Adds
axil5_master_rd_mon_cg	Both, with ready held low while gated
axil5_master_wr_mon_cg	Both, with ready held low while gated
axil5_slave_rd_mon_cg	Both, with ready held low while gated
axil5_slave_wr_mon_cg	Both, with ready held low while gated

1.6 Testing

val/amba/test_axil5_*.py – sixteen runners, 42 parameterised cases, all passing. They drive the modules with **every optional group enabled**, which makes them the only tests in the repo that exercise the AXI5-Lite sideband against real transport RTL rather than a behavioural model.

The testbench classes in bin/TBClasses/axil5/ are the AXI4-Lite ones with the component factories swapped and the group widths set in COMPONENT_KWARGS. One definition of the traffic and its checks.

Also here: test_axil5_opt_signals.py drives axil5_opt_slave, a behavioural slave under rtl/amba/axil5/test-modules/ built purely so the BFM's had a DUT carrying every optional port. It is test collateral – do not instantiate it in a design.

1.7 Navigation

[AMBA overview](#) | [AXI4-Lite](#) | [AXI5](#)

2 AXIL5 Master Read

Module: axil5_master_rd.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_master_rd

2.1 Overview

The AXIL5 Master Read provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_master_rd with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled

2.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	

Parameter	Type	Default	Description
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	$(DW / 64) > 0$? $(DW / 64) :$ 1	
ARSize	int	AW + 3 +	
RSize	int	DW + 2 +	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

2.3 Ports

Your logic drives the fub_* side; the m_axil_* side faces the bus.

Port	Direction	Width	Description
ack	Input	1	

Port	Direction	Width	Description
aresetn	Input	1	
fub_araddr	Input	[AW-1:0]	
fub_arprot	Input	[2:0]	
fub_arlock	Input	1	
fub_aruser	Input	[UW-1:0]	
fub_artrace	Input	1	
fub_arloop	Input	[LW-1:0]	
fub_arpam	Input	[MW-1:0]	
fub_armecid	Input	[EW-1:0]	
fub_arnsaid	Input	[NW-1:0]	
fub_arvalid	Input	1	
fub_arready	Output	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_arpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	

Port	Direction	Width	Description
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
fub_rdata	Output	[DW-1:0]	
fub_rresp	Output	[1:0]	
fub_ruser	Output	[UW-1:0]	
fub_rtrace	Output	1	
fub_rloop	Output	[LW-1:0]	
fub_rpoison	Output	[PW-1:0]	
fub_rvalid	Output	1	
fub_rready	Input	1	
busy	Output	1	

2.4 Functional Description

2.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

Group	Parameter	Signals on this module	Width
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

2.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

2.4.3 It transports; it does not interpret

MPAM, MECID, NSAIID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

2.5 Timing Characteristics

2.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR

Parameter	Default	Channel
SKID_DEPTH_R	4 entries	channel Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

2.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

2.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AR        (2),
```

```

        .SKID_DEPTH_R          (4)
    ) u_axil5_master_rd (
        // clock/reset
        .aclk                  (aclk),
        .aresetn                (aresetn),
        // fub_ar
        .fub_araddr             (fub_araddr),
        // ... `fub_arprot`, `fub_arlock`, `fub_aruser`, +7 more
        // m_axil_ar
        .m_axil_araddr          (m_axil_araddr),
        // ... `m_axil_arprot`, `m_axil_arlock`, `m_axil_aruser`, +7 more
        // m_axil_r
        .m_axil_rdata           (m_axil_rdata),
        // ... `m_axil_rresp`, `m_axil_ruser`, `m_axil_rtrace`, +4 more
        // fub_r
        .fub_rdata              (fub_rdata),
        // ... `fub_rresp`, `fub_ruser`, `fub_rtrace`, +4 more
        // status
        .busy                    (busy)
    );

```

2.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

2.8 Related Modules

- [axil5_master_rd_cg](#)

- [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil5_master_wr_mon](#)
 - [axil4_master_rd](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

2.9 Testing

`val/amba/test_axil5_master_rd.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

2.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

3 AXIL5 Master Read, Clock-Gated

Module: `axil5_master_rd_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_cg`

3.1 Overview

The AXIL5 Master Read, Clock-Gated provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_cg` with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)

- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- One `amba_clock_gate_ctrl` gating the whole inner module

3.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	

Parameter	Type	Default	Description
		TH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

3.3 Ports

Your logic drives the fub_* side; the m_axil_* side faces the bus. The cfg_cg_* inputs run the clock gate.

Port	Direction	Width	Description
acclk	Input	1	
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
fub_araddr	Input	[AW-1:0]	
fub_arprot	Input	[2:0]	
fub_arlock	Input	1	
fub_aruser	Input	[UW-1:0]	
fub_artrace	Input	1	
fub_arloop	Input	[LW-1:0]	
fub_arpam	Input	[MW-1:0]	
fub_armecid	Input	[EW-1:0]	

Port	Direction	Width	Description
fub_arnsaid	Input	[NW-1:0]	
fub_arvalid	Input	1	
fub_arready	Output	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_arpam	Output	[MW-1:0]	
m_axil_armeci	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
fub_rdata	Output	[DW-1:0]	
fub_rresp	Output	[1:0]	
fub_ruser	Output	[UW-1:0]	
fub_rtrace	Output	1	
fub_rloop	Output	[LW-1:0]	

Port	Direction	Width	Description
fub_rpoison	Output	[PW-1:0]	
fub_rvalid	Output	1	
fub_rready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

3.4 Functional Description

3.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

3.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

3.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

3.5 Timing Characteristics

3.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream

one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

3.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

3.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd_cg #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AR        (2),
    .SKID_DEPTH_R         (4),
    .CG_IDLE_COUNT_WIDTH  (4)
) u_axil5_master_rd_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // configuration
    .cfg_cg_enable        (cfg_cg_enable),
    .cfg_cg_idle_count    (cfg_cg_idle_count),
    // fub_ar
```

```

.fub_araddr      (fub_araddr),
// ... `fub_arprot`, `fub_arlock`, `fub_aruser`, +7 more
// m_axil_ar
.m_axil_araddr   (m_axil_araddr),
// ... `m_axil_arprot`, `m_axil_arlock`, `m_axil_aruser`, +7 more
// m_axil_r
.m_axil_rdata    (m_axil_rdata),
// ... `m_axil_rresp`, `m_axil_ruser`, `m_axil_rtrace`, +4 more
// fub_r
.fub_rdata      (fub_rdata),
// ... `fub_rresp`, `fub_ruser`, `fub_rtrace`, +4 more
// status
.cg_gating      (cg_gating),
.cg_idle       (cg_idle)
);

```

3.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer’s READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into user_valid pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. val/amba/test_cg_peer_ready.py parks the peer READY high, holds every VALID low, and requires cg_gating – see vault/handbook/design/clock-gating-activity-terms.md.

busy is not brought out here. The plain _cg wrappers consume the base module’s busy internally as a wake term; use cg_idle or cg_gating for idle detection. The _mon_cg variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only. With it low the clock free-runs and the module behaves exactly like its base.

3.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil5_master_wr_mon](#)
 - [axil4_master_rd_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

3.9 Testing

`val/amba/test_axil5_master_rd_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – TBClasses/axil5 sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

3.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

4 AXI5 Master Read Monitor

Module: `axil5_master_rd_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_mon`

4.1 Overview

The AXI5 Master Read Monitor provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_mon` with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
-

4.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_master_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_master_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_master_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_master_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Leave at 0 on AXI4-Lite. The wrapper hardwires cmd_id/data_id/resp_id to 1'b0, so enabling this with ID_MATCH_BASE above 0 makes id_owned(0) false for every

			transaction and drops ALL monitoring.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE =1. Must stay 0 on AXI4-Lite – every transaction reports ID 0.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
USER_WIDTH	int	4	

LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000A	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h01	Unit identifier emitted in the

unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

4.3 Ports

Your logic drives the fub_axil_* side; the m_axil_* side faces the bus. The cfg_* and monbus_* ports belong to the monitor.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
fub_axil_araddr	Input	[AW-1:0]	
fub_axil_arprot	Input	[2:0]	
fub_axil_arlock	Input	1	
fub_axil_aruser	Input	[UW-1:0]	
fub_axil_artrace	Input	1	
fub_axil_arloop	Input	[LW-1:0]	
fub_axil_arpam	Input	[MW-1:0]	

Port	Direction	Width	Description
fub_axil_armecid	Input	[EW-1:0]	
fub_axil_arnsaid	Input	[NW-1:0]	
fub_axil_arvalid	Input	1	
fub_axil_arready	Output	1	
fub_axil_rdata	Output	[DW-1:0]	
fub_axil_rresp	Output	[1:0]	
fub_axil_ruser	Output	[UW-1:0]	
fub_axil_rtrace	Output	1	
fub_axil_rloop	Output	[LW-1:0]	
fub_axil_rpoison	Output	[PW-1:0]	
fub_axil_rvalid	Output	1	
fub_axil_rready	Input	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_arpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	

Port	Direction	Width	Description
m_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event

Port	Direction	Width	Description
			mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	$[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1) - 1 : 0]$	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.

Port	Direction	Width	Description
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_read_y	Output	1	
cfg_conflict_error	Output	1	Configuration

Port	Direction	Width	Description
			conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

4.4 Functional Description

4.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH

Group	Parameter	Signals on this module	Width
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

4.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

4.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

4.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1.
`axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

4.6 Timing Characteristics

4.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the un stalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

4.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

4.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd_mon #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .ACLK_MHZ           (100),
    .USE_MONITOR        (1'b1),
    .N_ADDR_RANGES      (0),
    .UNIT_ID            (8'h01),
    .AGENT_ID           (16'h000A),
    .MAX_TRANSACTIONS   (8),
    .USE_WDATA_ORDER_Q  (1'b0),
    .NUM_BANKS          (1),
    .ID_FILTER_ENABLE   (1'b0),
    .ADDR_FILTER_ENABLE (1'b0),
    .ID_MATCH_BASE      (0),
    .ID_MATCH_COUNT     (0),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .USER_WIDTH          (4),
    .LOOP_WIDTH          (3),
    .MPAM_WIDTH          (11),
    .MECID_WIDTH         (16),
    .NSAID_WIDTH         (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_LOOP         (1),
    .ENABLE_MPAM         (1),
    .ENABLE_MECID        (1),
    .ENABLE_NSAID        (1),
    .ENABLE_POISON       (1),
    .ENABLE_LOCK         (1)
) u_axil5_master_rd_mon (
```

```

// clock/reset
.aclk                (aclk),
.aresetn             (aresetn),
// status
.cam_clear           (cam_clear),
// ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+31 more
// m_axil_ar
.m_axil_araddr       (m_axil_araddr),
// ... `m_axil_arprot`, `m_axil_arlock`, `m_axil_aruser`, +7 more
// m_axil_r
.m_axil_rdata        (m_axil_rdata),
// ... `m_axil_rresp`, `m_axil_ruser`, `m_axil_rtrace`, +4 more
// configuration
.cfg_monitor_enable  (cfg_monitor_enable),
// ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
// monitor bus
.monbus_valid        (monbus_valid)
// ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);

```

4.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil5_master_wr_mon](#)
 - axil4_master_rd_mon – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

4.9 Testing

val/amba/test_axil5_master_rd_mon.py drives this module with the AXI5-Lite BFM and **every optional group enabled** – TBClasses/axil5 sets the group widths in COMPONENT_KWARGS to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

4.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

5 AXI5 Master Read Monitor, Clock-Gated

Module: `axil5_master_rd_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_mon_cg`

5.1 Overview

The AXI5 Master Read Monitor, Clock-Gated provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_mon_cg` with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
 - Monitor plus clock gating; ready signals held low while gated
-

5.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon; outputs tied
N_ADDR_RANGES	int	0	0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000A	
			Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-

Parameter	Type	Default	Description
UNIT_ID		8'h01	bit Agent ID for monitor packets) Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

5.3 Ports

Your logic drives the fub_axil_* side; the m_axil_* side faces the bus. The cfg_* and monbus_* ports belong to the monitor, and cfg_cg_* runs the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
fub_axil_araddr	Input	[AW-1:0]	
fub_axil_arprot	Input	[2:0]	
fub_axil_arlock	Input	1	

Port	Direction	Width	Description
fub_axil_aruser	Input	[UW-1:0]	
fub_axil_artrace	Input	1	
fub_axil_arloop	Input	[LW-1:0]	
fub_axil_arpam	Input	[MW-1:0]	
fub_axil_armecid	Input	[EW-1:0]	
fub_axil_arnsaid	Input	[NW-1:0]	
fub_axil_arvalid	Input	1	
fub_axil_arready	Output	1	
fub_axil_rdata	Output	[DW-1:0]	
fub_axil_rresp	Output	[1:0]	
fub_axil_ruser	Output	[UW-1:0]	
fub_axil_rtrace	Output	1	
fub_axil_rloop	Output	[LW-1:0]	
fub_axil_rpoison	Output	[PW-1:0]	
fub_axil_rvalid	Output	1	
fub_axil_rready	Input	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_arpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	

Port	Direction	Width	Description
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for

Port	Direction	Width	Description
			packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	

Port	Direction	Width	Description
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.

Port	Direction	Width	Description
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

5.4 Functional Description

5.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

5.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH` = `AXIL_DATA_WIDTH` = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

5.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

5.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1.
`axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

5.6 Timing Characteristics

5.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R

Parameter	Default	Channel
		channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

5.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

5.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd_mon_cg #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .USE_MONITOR        (1'b1),
    .N_ADDR_RANGES      (0),
    .ADDR_FILTER_ENABLE (1'b0),
    .ACLK_MHZ           (100),
    .USE_WDATA_ORDER_Q  (1'b0),
    .NUM_BANKS          (1),
    .UNIT_ID            (8'h01),
    .AGENT_ID           (16'h000A),
    .MAX_TRANSACTIONS   (8),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
```

```

.ENABLE_PERF_LOGIC    (1'b1),
.ENABLE_DEBUG_LOGIC   (1'b0),
.CG_IDLE_COUNT_WIDTH (4),
.USER_WIDTH           (4),
.LOOP_WIDTH           (3),
.MPAM_WIDTH           (11),
.MECID_WIDTH          (16),
.NSAID_WIDTH          (4),
.ENABLE_USER          (1),
.ENABLE_TRACE         (1),
.ENABLE_LOOP          (1),
.ENABLE_MPAM          (1),
.ENABLE_MECID         (1),
.ENABLE_NSAID         (1),
.ENABLE_POISON        (1),
.ENABLE_LOCK          (1)
) u_axil5_master_rd_mon_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+32 more
    // m_axil_ar
    .m_axil_araddr       (m_axil_araddr),
    // ... `m_axil_arprot`, `m_axil_arlock`, `m_axil_aruser`, +7 more
    // m_axil_r
    .m_axil_rdata        (m_axil_rdata),
    // ... `m_axil_rresp`, `m_axil_ruser`, `m_axil_rtrace`, +4 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +30 more
    // monitor bus
    .monbus_valid        (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);

```

5.8 Related Modules

- [axil5_master_rd](#)
- [axil5_master_rd_cg](#)
- [axil5_master_rd_mon](#)
- [axil5_master_wr](#)

- [axil5_master_wr_cg](#)
 - [axil5_master_wr_mon](#)
 - [axil4_master_rd_mon_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

5.9 Testing

`val/amba/test_axil5_master_rd_mon_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

5.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

6 AXI5-Lite Master Read with Lite Monitor

Module: `axil5_master_rd_monlite.sv` **Location:** `rtl/amba/axil5/` **Status:** Production Ready
(`amba/monitor-lite` TASK-001, 2026-09-26)

6.1 Overview

`axil5_master_rd_monlite` is `axil5_master_rd` with `axi_monitor_lite` watching its master-side read channels. It is the lite-monitor sibling of `axil5_master_rd_mon`: same core, same taps, same 128-bit `monitor_packet_t` on the same monbus with the same `UNIT_ID/AGENT_ID`, so the monbus arbiter, the AXI-Lite group, the tally and the host tooling cannot tell the two apart. The difference is what it costs and what it can be asked for: about a fifth of the monitor LUTs (677 against 3,249 per read monitor in the same bridge fixture, see [axi_monitor_lite](#)), and only the four packet classes the lite emits: error, timeout, completion and active-count threshold.

6.1.1 What is not here

Compared with axil5_master_rd_mon: no performance window or counters, no debug packets, no address-range checker, no ID or address filters, no per-event masks, no latency threshold, and no block_ready. The lite never stalls the port: an event it cannot deliver is dropped and counted (dropped_count, reported as an EVENT_DROPPED packet), and a command that finds no free table entry is counted (refused_count) and left untracked, so its beats report as orphans.

6.1.2 Key Features

- Every axil5_master_rd parameter and port, declared verbatim and passed through by name
 - axi_monitor_lite on the m_-side taps; USE_MONITOR=0 omits it and ties its outputs
 - Nine config inputs: the enables, the microsecond timeout, the tick LUT index and the packet-type mask
 - Five status outputs, two of them (dropped_count, refused_count) unique to the lite
-

6.2 Parameters

6.2.1 Monitor Parameters

Parameter	Type	Default	Description
USE_MONITOR	bit	1'b1	0 = omit the monitor, tie its outputs
UNIT_ID	logic [7:0]	8'h01	Unit id in every packet
AGENT_ID	logic [15:0]	16'h000A	Agent id in every packet
MAX_TRANSACTIONS	int	8	table entries; a command finding none is counted, not tracked
ACTIVE_TRANS_THR ESHOLD	int	MAX_TRANSACTIONS / 2	Table occupancy that fires a threshold packet (rising edge)

Parameter	Type	Default	Description
OUT_DEPTH	int	4	monbus output queue, a power of two
ACLK_MHZ	int	100	Clock in MHz; keeps the 1 us tick exact
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT lower bound (cfg_freq_sel indexes it)
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT upper bound

6.2.2 Core Parameters (passed through to axil5_master_rd)

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	as on axil5_master_rd
AXIL_DATA_WIDTH	int	32	as on axil5_master_rd
USER_WIDTH	int	4	as on axil5_master_rd
LOOP_WIDTH	int	3	as on axil5_master_rd
MPAM_WIDTH	int	11	as on axil5_master_rd
MECID_WIDTH	int	16	as on axil5_master_rd
NSAID_WIDTH	int	4	as on axil5_master_rd
ENABLE_USER	bit	1	as on axil5_master_rd
ENABLE_TRACE	bit	1	as on axil5_master_rd
ENABLE_LOOP	bit	1	as on

Parameter	Type	Default	Description
ENABLE_MPAM	bit	1	axil5_master_rd as on axil5_master_rd
ENABLE_MECID	bit	1	as on axil5_master_rd
ENABLE_NSAID	bit	1	as on axil5_master_rd
ENABLE_POISON	bit	1	as on axil5_master_rd
ENABLE_LOCK	bit	1	as on axil5_master_rd
SKID_DEPTH_AR	int	2	as on axil5_master_rd
SKID_DEPTH_R	int	4	as on axil5_master_rd
AW	int	AXIL_ADDR_WIDTH	as on axil5_master_rd
DW	int	AXIL_DATA_WIDTH	as on axil5_master_rd
UW	int	USER_WIDTH	as on axil5_master_rd
LW	int	LOOP_WIDTH	as on axil5_master_rd
MW	int	MPAM_WIDTH	as on axil5_master_rd
EW	int	MECID_WIDTH	as on axil5_master_rd
NW	int	NSAID_WIDTH	as on axil5_master_rd
PW	int	$(DW / 64) > 0 ?$ $(DW / 64) : 1$	as on axil5_master_rd
ARSize	int	$AW + 3 +$	as on axil5_master_rd
RSize	int	$DW + 2 +$	as on axil5_master_rd

6.3 Ports

6.3.1 Monitor Control, Bus and Status

Port	Direction	Type	Description
cam_clear	input	logic	sync clear of the monitor's table (named as on the _mon wrapper: pin-compatible)
cfg_monitor_enable	input	logic	0 = monitor inert, table held clear
cfg_error_enable	input	logic	
cfg_timeout_enable	input	logic	
cfg_compl_enable	input	logic	
cfg_threshold_enable	input	logic	
cfg_timeout_cycles	input	logic [15:0]	MICROSECONDS of no progress; 0 = never
cfg_freq_sel	input	logic [3:0]	counter_freq_invariant LUT index
cfg_axi_pkt_mask	input	logic [15:0]	drop mask by packet type
i_mon_time	input	monitor_common_pkg::monbus_timestamp_t	
monbus_valid	output	logic	
monbus_ready	input	logic	
monbus_packet	output	monitor_common_pkg::monitor_packet_t	
monbus_timestamp	output	monitor_common_pkg::monbus_timestamp_t	
active_transactions	output	logic [7:0]	entries in the table

Port	Direction	Type	Description
error_count	output	logic [15:0]	error + timeout packets emitted
transaction_count	output	logic [31:0]	completion packets emitted
dropped_count	output	logic [15:0]	events lost to monbus backpressure since the last report
refused_count	output	logic [15:0]	commands that found no free table entry

6.3.2 Core Ports (as on axil5_master_rd)

Port	Direction	Type	Description
aclk	input	logic	
aresetn	input	logic	
fub_axil_araddr	input	logic [AW-1:0]	
fub_axil_arprot	input	logic [2:0]	
fub_axil_arlock	input	logic	
fub_axil_aruser	input	logic [UW-1:0]	
fub_axil_artrace	input	logic	
fub_axil_arlockop	input	logic [LW-1:0]	
fub_axil_armpam	input	logic [MW-1:0]	
fub_axil_armecid	input	logic [EW-1:0]	
fub_axil_arnsaids	input	logic [NW-1:0]	
fub_axil_arvalid	input	logic	

Port	Direction	Type	Description
fub_axil_arready	output	logic	
m_axil_araddr	output	logic [AW-1:0]	
m_axil_arprot	output	logic [2:0]	
m_axil_arlock	output	logic	
m_axil_aruser	output	logic [UW-1:0]	
m_axil_artrace	output	logic	
m_axil_arloop	output	logic [LW-1:0]	
m_axil_armspam	output	logic [MW-1:0]	
m_axil_armeci	output	logic [EW-1:0]	
m_axil_arnsai	output	logic [NW-1:0]	
m_axil_arvalid	output	logic	
m_axil_arready	input	logic	
m_axil_rdata	input	logic [DW-1:0]	
m_axil_rresp	input	logic [1:0]	
m_axil_ruser	input	logic [UW-1:0]	
m_axil_rtrace	input	logic	
m_axil_rloop	input	logic [LW-1:0]	
m_axil_rpoison	input	logic [PW-1:0]	
m_axil_rvalid	input	logic	
m_axil_rready	output	logic	
fub_axil_rdata	output	logic [DW-1:0]	
fub_axil_rresp	output	logic [1:0]	
fub_axil_ruse	output	logic [UW-	

Port	Direction	Type	Description
r		1:0]	
fub_axil_rtra	output	logic	
ce			
fub_axil_rloop	output	logic [LW-1:0]	
fub_axil_rposition	output	logic [PW-1:0]	
fub_axil_rvalid	output	logic	
fub_axil_rready	input	logic	
busy	output	logic	

6.4 Functional Description

The core is instantiated untouched; unlike `axil5_master_rd_mon` there is no gating of the command handshake, because the lite has no admission stall. The monitor taps the m_-side command, data and response channels through three valids gated by `cfg_monitor_enable`, which also holds the table clear while low. `cfg_timeout_cycles` is a microsecond count of no progress on an entry, passed at full width; 0 means never. The active-count threshold is the parameter `ACTIVE_TRANS_THRESHOLD`, as on the `_mon` wrapper.

The lite's own behaviour – per-ID linked-list attribution, stamps instead of counters, the registered event stage, the 4-entry output queue and drop-and-count – is on the [axi_monitor_lite](#) page.

6.5 Timing Characteristics

The datapath timing is `axil5_master_rd`'s. The monitor adds no gating to any handshake and no path into the datapath; its own paths are two short cycles (attribution, then packet formatting) and met 10 ns on an Artix-7 100T -1 with margin inside `bridge_1x2_rd_lite_mon`.

6.6 Usage Examples

```
axil5_master_rd_monlite #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
```

```

        .UNIT_ID                (8'h02),
        .AGENT_ID               (16'h0014),
        .MAX_TRANSACTIONS       (8)
    ) u_axil5_master_rd_monlite (
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // ... the axil5_master_rd ports, by name ...
        .cam_clear               (1'b0),
        .cfg_monitor_enable      (1'b1),
        .cfg_error_enable        (1'b1),
        .cfg_timeout_enable      (1'b1),
        .cfg_compl_enable        (1'b0),
        .cfg_threshold_enable    (1'b0),
        .cfg_timeout_cycles      (16'd1000),
        .cfg_freq_sel            (4'd0),
        .cfg_axi_pkt_mask        (16'h0000),
        .i_mon_time              (mon_time),
        .monbus_valid            (monbus_valid),
        .monbus_ready            (monbus_ready),
        .monbus_packet           (monbus_packet),
        .monbus_timestamp        (monbus_timestamp),
        .active_transactions     (),
        .error_count             (),
        .transaction_count       (),
        .dropped_count           (dropped_count),
        .refused_count           (refused_count)
    );

```

In a generated bridge, `mon_preset = "lite"` selects this wrapper for every monitored port.

6.7 Design Notes

- `cam_clear` keeps the `_mon` wrapper's name so the two are pin-compatible on the monitor side; the lite has a table, not a CAM.
 - `MAX_TRANSACTIONS` defaults to 8 here against 16 on `axil5_master_rd_mon`: size it to the port's real outstanding depth, since refused commands surface as orphans.
 - Consumers that relied on `block_ready` to bound the table get drop-and-count instead; the count is always reported, the identity of the lost events is not.
-

6.8 Related Modules

- `axil5_master_rd_mon` – the full-monitor sibling
 - `axil5_master_rd` – the core this wraps
 - [axi_monitor_lite](#) – the monitor itself
-

6.9 Testing

`val/amba/monitor-lite/test_axil5_master_rd_monlite.py` exercises this module: the same TB and scenarios as `val/amba/test_axil5_master_rd_mon.py`, with the DUT swapped.

```
source env_python
make -C val/amba/monitor-lite run-axil5_master_rd_monlite-gate
```

Last Updated: 2026-09-26

6.10 Navigation

- [← Back to Monitor Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)

7 AXIL5 Master Write

Module: `axil5_master_wr.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr`

7.1 Overview

The AXIL5 Master Write provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr` with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support

- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled

7.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	

Parameter	Type	Default	Description
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AWSize	int	AW + 3 +	
WSize	int	DW + SW +	
BSize	int	2 +	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

7.3 Ports

Your logic drives the fub_* side; the m_axil_* side faces the bus.

Port	Direction	Width	Description
ack	Input	1	
aresetn	Input	1	
fub_awaddr	Input	[AW-1:0]	
fub_awprot	Input	[2:0]	
fub_awlock	Input	1	
fub_awuser	Input	[UW-1:0]	
fub_awtrace	Input	1	
fub_awloop	Input	[LW-1:0]	
fub_awmpam	Input	[MW-1:0]	
fub_awmecid	Input	[EW-1:0]	
fub_awnsaid	Input	[NW-1:0]	
fub_awvalid	Input	1	

Port	Direction	Width	Description
fub_awready	Output	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmeci	Output	[EW-1:0]	
m_axil_awnsai	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
fub_wdata	Input	[DW-1:0]	
fub_wstrb	Input	[SW-1:0]	
fub_wuser	Input	[UW-1:0]	
fub_wpoison	Input	[PW-1:0]	
fub_wvalid	Input	1	
fub_wready	Output	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[SW-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	

Port	Direction	Width	Description
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
fub_bresp	Output	[1:0]	
fub_buser	Output	[UW-1:0]	
fub_btrace	Output	1	
fub_bloop	Output	[LW-1:0]	
fub_bvalid	Output	1	
fub_bready	Input	1	
busy	Output	1	

7.4 Functional Description

7.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

7.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

7.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

7.5 Timing Characteristics

7.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

7.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

7.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AW        (2),
    .SKID_DEPTH_W         (4),
    .SKID_DEPTH_B         (2)
) u_axil5_master_wr (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // fub_aw
```

```

.fub_awaddr      (fub_awaddr),
// ... `fub_awprot`, `fub_awlock`, `fub_awuser`, +7 more
// m_axil_aw
.m_axil_awaddr   (m_axil_awaddr),
// ... `m_axil_awprot`, `m_axil_awlock`, `m_axil_awuser`, +7 more
// fub_w
.fub_wdata       (fub_wdata),
// ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
// m_axil_w
.m_axil_wdata    (m_axil_wdata),
// ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
// m_axil_b
.m_axil_bresp    (m_axil_bresp),
// ... `m_axil_buser`, `m_axil_btrace`, `m_axil_bloop`, +2 more
// fub_b
.fub_bresp       (fub_bresp),
// ... `fub_buser`, `fub_btrace`, `fub_bloop`, +2 more
// status
.busy            (busy)
);

```

7.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFM's against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

7.8 Related Modules

- [axil5_master_rd](#)

- [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr_cg](#)
 - [axil5_master_wr_mon](#)
 - [axil4_master_wr](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

7.9 Testing

`val/amba/test_axil5_master_wr.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

7.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

8 AXIL5 Master Write, Clock-Gated

Module: `axil5_master_wr_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_cg`

8.1 Overview

The AXIL5 Master Write, Clock-Gated provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_cg` with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)

- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- One `amba_clock_gate_ctrl` gating the whole inner module

8.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	

Parameter	Type	Default	Description
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

8.3 Ports

Your logic drives the fub_* side; the m_axil_* side faces the bus. The cfg_cg_* inputs run the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
fub_awaddr	Input	[AW-1:0]	
fub_awprot	Input	[2:0]	
fub_awlock	Input	1	
fub_awuser	Input	[UW-1:0]	
fub_awtrace	Input	1	
fub_awloop	Input	[LW-1:0]	
fub_awmpam	Input	[MW-1:0]	

Port	Direction	Width	Description
fub_awmecid	Input	[EW-1:0]	
fub_awnsaid	Input	[NW-1:0]	
fub_awvalid	Input	1	
fub_awready	Output	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrac e	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmeci d	Output	[EW-1:0]	
m_axil_awnsai d	Output	[NW-1:0]	
m_axil_awvali d	Output	1	
m_axil_awread y	Input	1	
fub_wdata	Input	[DW-1:0]	
fub_wstrb	Input	[SW-1:0]	
fub_wuser	Input	[UW-1:0]	
fub_wpoison	Input	[PW-1:0]	
fub_wvalid	Input	1	
fub_wready	Output	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[SW-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoiso n	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	

Port	Direction	Width	Description
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
fub_bresp	Output	[1:0]	
fub_buser	Output	[UW-1:0]	
fub_btrace	Output	1	
fub_bloop	Output	[LW-1:0]	
fub_bvalid	Output	1	
fub_bready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

8.4 Functional Description

8.4.1 Optional signal groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH

Group	Parameter	Signals on this module	Width
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

8.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

8.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

8.5 Timing Characteristics

8.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

8.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

8.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr_cg #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH            (4),
    .LOOP_WIDTH            (3),
    .MPAM_WIDTH            (11),
    .MECID_WIDTH           (16),
    .NSAID_WIDTH           (4),
    .ENABLE_USER           (1),
```

```

.ENABLE_TRACE          (1),
.ENABLE_LOOP           (1),
.ENABLE_MPAM           (1),
.ENABLE_MECID          (1),
.ENABLE_NSAID          (1),
.ENABLE_POISON         (1),
.ENABLE_LOCK           (1),
.SKID_DEPTH_AW         (2),
.SKID_DEPTH_W          (4),
.SKID_DEPTH_B          (2),
.CG_IDLE_COUNT_WIDTH   (4)
) u_axil5_master_wr_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // configuration
    .cfg_cg_enable        (cfg_cg_enable),
    .cfg_cg_idle_count    (cfg_cg_idle_count),
    // fub_aw
    .fub_awaddr           (fub_awaddr),
    // ... `fub_awprot`, `fub_awlock`, `fub_awuser`, +7 more
    // m_axil_aw
    .m_axil_awaddr        (m_axil_awaddr),
    // ... `m_axil_awprot`, `m_axil_awlock`, `m_axil_awuser`, +7 more
    // fub_w
    .fub_wdata            (fub_wdata),
    // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
    // m_axil_w
    .m_axil_wdata         (m_axil_wdata),
    // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
    // m_axil_b
    .m_axil_bresp         (m_axil_bresp),
    // ... `m_axil_buser`, `m_axil_btrace`, `m_axil_bloop`, +2 more
    // fub_b
    .fub_bresp            (fub_bresp),
    // ... `fub_buser`, `fub_btrace`, `fub_bloop`, +2 more
    // status
    .cg_gating            (cg_gating),
    .cg_idle              (cg_idle)
);

```

8.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by

running AXI5-Lite BFM's against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The `*Size` parameters and the `AW/DW` aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer’s READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating` – see `vault/handbook/design/clock-gating-activity-terms.md`.

busy is not brought out here. The plain `_cg` wrappers consume the base module’s busy internally as a wake term; use `cg_idle` or `cg_gating` for idle detection. The `_mon_cg` variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only. With it low the clock free-runs and the module behaves exactly like its base.

8.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_mon](#)
 - `axil4_master_wr_cg` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

8.9 Testing

`val/amba/test_axil5_master_wr_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

8.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

9 AXI5 Master Write Monitor

Module: `axil5_master_wr_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_mon`

9.1 Overview

The AXI5 Master Write Monitor provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_mon` with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
-

9.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_master_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_master_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_master_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_master_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	2	
SKID_DEPTH_B	int	2	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled

Parameter	Type	Default	Description
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Leave at 0 on AXI4-Lite. The wrapper hardwires cmd_id/data_id/resp_id to 1'b0, so enabling this with ID_MATCH_BASE above 0 makes id_owned(0) false for every transaction and drops ALL monitoring.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE =1. Must stay 0 on AXI4-Lite – every transaction reports ID 0.

Parameter	Type	Default	Description
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THR ESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	

Parameter	Type	Default	Description
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000B	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h01	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

9.3 Ports

Your logic drives the fub_axil_* side; the m_axil_* side faces the bus. The cfg_* and monbus_* ports belong to the monitor.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
fub_axil_awaddr	Input	[AW-1:0]	
fub_axil_awprot	Input	[2:0]	
fub_axil_awlock	Input	1	
fub_axil_awuser	Input	[UW-1:0]	
fub_axil_awtrace	Input	1	
fub_axil_awloop	Input	[LW-1:0]	
fub_axil_awmpam	Input	[MW-1:0]	
fub_axil_awmecid	Input	[EW-1:0]	
fub_axil_awnsaid	Input	[NW-1:0]	
fub_axil_awvalid	Input	1	
fub_axil_awready	Output	1	
fub_axil_wdata	Input	[DW-1:0]	
fub_axil_wstrb	Input	[DW/8-1:0]	
fub_axil_wuser	Input	[UW-1:0]	
fub_axil_wpoison	Input	[PW-1:0]	
fub_axil_wvalid	Input	1	
fub_axil_wready	Output	1	

Port	Direction	Width	Description
fub_axil_bresp	Output	[1:0]	
fub_axil_buser	Output	[UW-1:0]	
fub_axil_btrace	Output	1	
fub_axil_bloop	Output	[LW-1:0]	
fub_axil_bvalid	Output	1	
fub_axil_bready	Input	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmecid	Output	[EW-1:0]	
m_axil_awnsaid	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[DW/8-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable

Port	Direction	Width	Description
cfg_error_enable	Input	1	monitoring Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual

Port	Direction	Width	Description
sk			completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	$[(N_ADDR_RANGES > 0) ? N_ADDR_RANGES : 1] - 1 : 0]$	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for

Port	Direction	Width	Description
i_mon_time	Input	monbus_timestamp_t	the packet on monbus_packet. Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	

Port	Direction	Width	Description
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

9.4 Functional Description

9.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH

Group	Parameter	Signals on this module	Width
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

9.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

9.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

9.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1.
`axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

9.6 Timing Characteristics

9.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

9.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

9.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr_mon #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (2),
    .SKID_DEPTH_B       (2),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .ACLK_MHZ           (100),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES      (0),
    .UNIT_ID             (8'h01),
    .AGENT_ID            (16'h000B),
    .MAX_TRANSACTIONS    (8),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .ID_FILTER_ENABLE    (1'b0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ID_MATCH_BASE       (0),
    .ID_MATCH_COUNT      (0),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .USER_WIDTH          (4),
    .LOOP_WIDTH          (3),
    .MPAM_WIDTH          (11),
    .MECID_WIDTH         (16),
    .NSAID_WIDTH         (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
```

```

        .ENABLE_LOOP            (1),
        .ENABLE_MPAM            (1),
        .ENABLE_MECID           (1),
        .ENABLE_NSaid           (1),
        .ENABLE_POISON          (1),
        .ENABLE_LOCK            (1)
    ) u_axil5_master_wr_mon (
        // clock/reset
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // status
        .cam_clear               (cam_clear),
        // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+35 more
        // m_axil_aw
        .m_axil_awaddr           (m_axil_awaddr),
        // ... `m_axil_awprot`, `m_axil_awlock`, `m_axil_awuser`, +7 more
        // m_axil_w
        .m_axil_wdata            (m_axil_wdata),
        // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
        // m_axil_b
        .m_axil_bresp            (m_axil_bresp),
        // ... `m_axil_buser`, `m_axil_btrace`, `m_axil_bloop`, +2 more
        // configuration
        .cfg_monitor_enable      (cfg_monitor_enable),
        // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
        // monitor bus
        .monbus_valid            (monbus_valid)
        // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
    );

```

9.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_master_wr_mon](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

9.9 Testing

`val/amba/test_axil5_master_wr_mon.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

9.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

10 AXIL5 Master Write Monitor, Clock-Gated

Module: `axil5_master_wr_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_mon_cg`

10.1 Overview

The AXIL5 Master Write Monitor, Clock-Gated provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_mon_cg` with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
 - Monitor plus clock gating; ready signals held low while gated
-

10.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	2	
SKID_DEPTH_B	int	2	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon; outputs tied
N_ADDR_RANGES	int	0	0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	

Parameter	Type	Default	Description
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000B	Agent identifier emitted in the agent_id field of

Parameter	Type	Default	Description
UNIT_ID		8'h01	every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets) Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

10.3 Ports

Your logic drives the fub_axil_* side; the m_axil_* side faces the bus. The cfg_* and monbus_* ports belong to the monitor, and cfg_cg_* runs the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the

Port	Direction	Width	Description
			monitor trans CAM
fub_axil_awaddr	Input	[AW-1:0]	
fub_axil_awprot	Input	[2:0]	
fub_axil_awlock	Input	1	
fub_axil_awuser	Input	[UW-1:0]	
fub_axil_awtrace	Input	1	
fub_axil_awloop	Input	[LW-1:0]	
fub_axil_awmpam	Input	[MW-1:0]	
fub_axil_awmecid	Input	[EW-1:0]	
fub_axil_awnsaid	Input	[NW-1:0]	
fub_axil_awvalid	Input	1	
fub_axil_awready	Output	1	
fub_axil_wdata	Input	[DW-1:0]	
fub_axil_wstrb	Input	[DW/8-1:0]	
fub_axil_wuser	Input	[UW-1:0]	
fub_axil_wpoison	Input	[PW-1:0]	
fub_axil_wvalid	Input	1	
fub_axil_wready	Output	1	
fub_axil_bresp	Output	[1:0]	
fub_axil_buser	Output	[UW-1:0]	
fub_axil_btrace	Output	1	
fub_axil_bloop	Output	[LW-1:0]	
fub_axil_bvalid	Output	1	
fub_axil_bready	Input	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	

Port	Direction	Width	Description
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmecid	Output	[EW-1:0]	
m_axil_awnsaid	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[DW/8-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold

Port	Direction	Width	Description
cfg_debug_enable	Input	1	packets Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug

Port	Direction	Width	Description
sk			event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_W IDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1) - 1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against

Port	Direction	Width	Description
			one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	

Port	Direction	Width	Description
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

10.4 Functional Description

10.4.1 Optional signal groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

10.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

10.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

10.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A **filter parameter only decides whether the logic is SYNTHESISED**. A build that sets ADDR_FILTER_ENABLE but leaves cfg_addr_filter_enable low filters nothing and looks broken. The parameter and the runtime port are both required.

10.6 Timing Characteristics

10.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

10.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, ASize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

10.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr_mon_cg #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (2),
```

```

.SKID_DEPTH_B          (2),
.AXIL_ADDR_WIDTH       (32),
.AXIL_DATA_WIDTH       (32),
.USE_MONITOR           (1'b1),
.N_ADDR_RANGES         (0),
.ADDR_FILTER_ENABLE    (1'b0),
.ACLK_MHZ              (100),
.USE_WDATA_ORDER_Q     (1'b0),
.NUM_BANKS             (1),
.UNIT_ID               (8'h01),
.AGENT_ID               (16'h000B),
.MAX_TRANSACTIONS      (8),
.ENABLE_FILTERING      (1),
.ADD_PIPELINE_STAGE    (0),
.ENABLE_ERROR_LOGIC    (1'b1),
.ENABLE_TIMEOUT_LOGIC  (1'b1),
.ENABLE_COMPL_LOGIC    (1'b1),
.ENABLE_THRESHOLD_LOGIC(1'b1),
.ENABLE_PERF_LOGIC     (1'b1),
.ENABLE_DEBUG_LOGIC    (1'b0),
.CG_IDLE_COUNT_WIDTH   (4),
.USER_WIDTH            (4),
.LOOP_WIDTH            (3),
.MPAM_WIDTH            (11),
.MECID_WIDTH           (16),
.NSAID_WIDTH           (4),
.ENABLE_USER           (1),
.ENABLE_TRACE          (1),
.ENABLE_LOOP           (1),
.ENABLE_MPAM           (1),
.ENABLE_MECID          (1),
.ENABLE_NSAID          (1),
.ENABLE_POISON         (1),
.ENABLE_LOCK           (1)
) u_axil5_master_wr_mon_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // status
    .cam_clear            (cam_clear),
    // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+36 more
    // m_axil_aw
    .m_axil_awaddr        (m_axil_awaddr),
    // ... `m_axil_awprot`, `m_axil_awlock`, `m_axil_awuser`, +7 more
    // m_axil_w
    .m_axil_wdata         (m_axil_wdata),
    // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more

```

```

// m_axil_b
.m_axil_bresp      (m_axil_bresp),
// ... `m_axil_buser`, `m_axil_btrace`, `m_axil_bloop`, +2 more
// configuration
.cfg_monitor_enable (cfg_monitor_enable),
// ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +30 more
// monitor bus
.monbus_valid      (monbus_valid)
// ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);

```

10.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_master_wr_mon_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

10.9 Testing

`val/amba/test_axil5_master_wr_mon_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – TBClasses/axil5 sets the group widths in COMPONENT_KWARGS to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

10.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

11 AXI5-Lite Master Write with Lite Monitor

Module: axil5_master_wr_monlite.sv **Location:** rtl/amba/axil5/ **Status:** Production Ready (amba/monitor-lite TASK-001, 2026-09-26)

11.1 Overview

axil5_master_wr_monlite is axil5_master_wr with axi_monitor_lite watching its master-side write channels. It is the lite-monitor sibling of axil5_master_wr_mon: same core, same taps, same 128-bit monitor_packet_t on the same monbus with the same UNIT_ID/AGENT_ID, so the monbus arbiter, the AXI-Lite group, the tally and the host tooling cannot tell the two apart. The difference is what it costs and what it can be asked for: about a fifth of the monitor LUTs (677 against 3,249 per read monitor in the same bridge fixture, see [axi_monitor_lite](#)), and only the four packet classes the lite emits: error, timeout, completion and active-count threshold.

11.1.1 What is not here

Compared with axil5_master_wr_mon: no performance window or counters, no debug packets, no address-range checker, no ID or address filters, no per-event masks, no latency threshold, and no block_ready. The lite never stalls the port: an event it cannot deliver is dropped and counted (dropped_count, reported as an EVENT_DROPPED packet), and a command that finds no free table entry is counted (refused_count) and left untracked, so its beats report as orphans.

11.1.2 Key Features

- Every axil5_master_wr parameter and port, declared verbatim and passed through by name
 - axi_monitor_lite on the m_-side taps; USE_MONITOR=0 omits it and ties its outputs
 - Nine config inputs: the enables, the microsecond timeout, the tick LUT index and the packet-type mask
 - Five status outputs, two of them (dropped_count, refused_count) unique to the lite
-

11.2 Parameters

11.2.1 Monitor Parameters

Parameter	Type	Default	Description
USE_MONITOR	bit	1'b1	0 = omit the

Parameter	Type	Default	Description
UNIT_ID	logic [7:0]	8'h01	monitor, tie its outputs Unit id in every packet
AGENT_ID	logic [15:0]	16'h000B	Agent id in every packet
MAX_TRANSACTIONS	int	8	table entries; a command finding none is counted, not tracked
ACTIVE_TRANS_THR ESHOLD	int	MAX_TRANSACTIONS / 2	Table occupancy that fires a threshold packet (rising edge)
OUT_DEPTH	int	4	monbus output queue, a power of two
ACLK_MHZ	int	100	Clock in MHz; keeps the 1 us tick exact
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT lower bound (cfg_freq_sel indexes it)
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT upper bound

11.2.2 Core Parameters (passed through to axil5_master_wr)

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	as on axil5_master_wr
AXIL_DATA_WIDTH	int	32	as on axil5_master_wr

Parameter	Type	Default	Description
USER_WIDTH	int	4	as on axil5_master_wr
LOOP_WIDTH	int	3	as on axil5_master_wr
MPAM_WIDTH	int	11	as on axil5_master_wr
MECID_WIDTH	int	16	as on axil5_master_wr
NSAID_WIDTH	int	4	as on axil5_master_wr
ENABLE_USER	bit	1	as on axil5_master_wr
ENABLE_TRACE	bit	1	as on axil5_master_wr
ENABLE_LOOP	bit	1	as on axil5_master_wr
ENABLE_MPAM	bit	1	as on axil5_master_wr
ENABLE_MECID	bit	1	as on axil5_master_wr
ENABLE_NSAID	bit	1	as on axil5_master_wr
ENABLE_POISON	bit	1	as on axil5_master_wr
ENABLE_LOCK	bit	1	as on axil5_master_wr
SKID_DEPTH_AW	int	2	as on axil5_master_wr
SKID_DEPTH_W	int	4	as on axil5_master_wr
SKID_DEPTH_B	int	2	as on axil5_master_wr
AW	int	AXIL_ADDR_WIDTH	as on axil5_master_wr
DW	int	AXIL_DATA_WIDTH	as on axil5_master_wr
SW	int	DW / 8	as on

Parameter	Type	Default	Description
UW	int	USER_WIDTH	axil5_master_wr as on axil5_master_wr
LW	int	LOOP_WIDTH	as on axil5_master_wr
MW	int	MPAM_WIDTH	as on axil5_master_wr
EW	int	MECID_WIDTH	as on axil5_master_wr
NW	int	NSAID_WIDTH	as on axil5_master_wr
PW	int	$(DW / 64) > 0 ?$ $(DW / 64) : 1$	as on axil5_master_wr
AWSize	int	$AW + 3 +$	as on axil5_master_wr
WSize	int	$DW + SW +$	as on axil5_master_wr
BSize	int	$2 +$	as on axil5_master_wr

11.3 Ports

11.3.1 Monitor Control, Bus and Status

Port	Direction	Type	Description
cam_clear	input	logic	sync clear of the monitor's table (named as on the _mon wrapper: pin-compatible)
cfg_monitor_enable	input	logic	0 = monitor inert, table held clear
cfg_error_enable	input	logic	
cfg_timeout_enable	input	logic	
cfg_compl_enable	input	logic	

Port	Direction	Type	Description
cfg_threshold_enable	input	logic	
cfg_timeout_cycles	input	logic [15:0]	MICROSECONDS of no progress; 0 = never
cfg_freq_sel	input	logic [3:0]	counter_freq_invariant LUT index
cfg_axi_pkt_mask	input	logic [15:0]	drop mask by packet type
i_mon_time	input	monitor_common_pkg::monbus_timestamp_t	
monbus_valid	output	logic	
monbus_ready	input	logic	
monbus_packet	output	monitor_common_pkg::monitor_packet_t	
monbus_timestamp	output	monitor_common_pkg::monbus_timestamp_t	
active_transactions	output	logic [7:0]	entries in the table
error_count	output	logic [15:0]	error + timeout packets emitted
transaction_count	output	logic [31:0]	completion packets emitted
dropped_count	output	logic [15:0]	events lost to monbus backpressure since the last report
refused_count	output	logic [15:0]	commands that found no free table entry

11.3.2 Core Ports (as on axil5_master_wr)

Port	Direction	Type	Description
aclk	input	logic	
aresetn	input	logic	
fub_axil_awaddr	input	logic [AW-1:0]	
fub_axil_awprot	input	logic [2:0]	
fub_axil_awlock	input	logic	
fub_axil_awuser	input	logic [UW-1:0]	
fub_axil_awtrace	input	logic	
fub_axil_awloop	input	logic [LW-1:0]	
fub_axil_awmpam	input	logic [MW-1:0]	
fub_axil_awmecid	input	logic [EW-1:0]	
fub_axil_awnsaid	input	logic [NW-1:0]	
fub_axil_awvalid	input	logic	
fub_axil_awready	output	logic	
m_axil_awaddr	output	logic [AW-1:0]	
m_axil_awprot	output	logic [2:0]	
m_axil_awlock	output	logic	
m_axil_awuser	output	logic [UW-1:0]	
m_axil_awtrace	output	logic	
m_axil_awloop	output	logic [LW-1:0]	
m_axil_awmpam	output	logic [MW-1:0]	
m_axil_awmecid	output	logic [EW-1:0]	

Port	Direction	Type	Description
m_axil_awnsai d	output	logic [NW- 1:0]	
m_axil_awvali d	output	logic	
m_axil_awread y	input	logic	
fub_axil_wdat a	input	logic [DW- 1:0]	
fub_axil_wstr b	input	logic [SW- 1:0]	
fub_axil_wuse r	input	logic [UW- 1:0]	
fub_axil_wpoi son	input	logic [PW- 1:0]	
fub_axil_wval id	input	logic	
fub_axil_wrea dy	output	logic	
m_axil_wdata	output	logic [DW- 1:0]	
m_axil_wstrb	output	logic [SW- 1:0]	
m_axil_wuser	output	logic [UW- 1:0]	
m_axil_wpoiso n	output	logic [PW- 1:0]	
m_axil_wvalid	output	logic	
m_axil_wready	input	logic	
m_axil_bresp	input	logic [1:0]	
m_axil_buser	input	logic [UW- 1:0]	
m_axil_btrace	input	logic	
m_axil_bloop	input	logic [LW- 1:0]	
m_axil_bvalid	input	logic	
m_axil_bready	output	logic	
fub_axil_bres p	output	logic [1:0]	
fub_axil_buse	output	logic [UW-	

Port	Direction	Type	Description
r		1:0]	
fub_axil_btra	output	logic	
ce			
fub_axil_bloop	output	logic [LW-1:0]	
fub_axil_bvalid	output	logic	
fub_axil_bready	input	logic	
busy	output	logic	

11.4 Functional Description

The core is instantiated untouched; unlike `axil5_master_wr_mon` there is no gating of the command handshake, because the lite has no admission stall. The monitor taps the m_-side command, data and response channels through three valids gated by `cfg_monitor_enable`, which also holds the table clear while low. `cfg_timeout_cycles` is a microsecond count of no progress on an entry, passed at full width; 0 means never. The active-count threshold is the parameter `ACTIVE_TRANS_THRESHOLD`, as on the `_mon` wrapper.

The lite's own behaviour – per-ID linked-list attribution, stamps instead of counters, the registered event stage, the 4-entry output queue and drop-and-count – is on the [axi_monitor_lite](#) page.

11.5 Timing Characteristics

The datapath timing is `axil5_master_wr`'s. The monitor adds no gating to any handshake and no path into the datapath; its own paths are two short cycles (attribution, then packet formatting) and met 10 ns on an Artix-7 100T -1 with margin inside `bridge_1x2_rd_lite_mon`.

11.6 Usage Examples

```
axil5_master_wr_monlite #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .UNIT_ID              (8'h02),
    .AGENT_ID             (16'h0014),
```

```

        .MAX_TRANSACTIONS      (8)
    ) u_axil5_master_wr_monlite (
        .aclk                  (aclk),
        .aresetn               (aresetn),
        // ... the axil5_master_wr ports, by name ...
        .cam_clear              (1'b0),
        .cfg_monitor_enable     (1'b1),
        .cfg_error_enable       (1'b1),
        .cfg_timeout_enable     (1'b1),
        .cfg_compl_enable       (1'b0),
        .cfg_threshold_enable   (1'b0),
        .cfg_timeout_cycles     (16'd1000),
        .cfg_freq_sel           (4'd0),
        .cfg_axi_pkt_mask       (16'h0000),
        .i_mon_time             (mon_time),
        .monbus_valid           (monbus_valid),
        .monbus_ready           (monbus_ready),
        .monbus_packet          (monbus_packet),
        .monbus_timestamp       (monbus_timestamp),
        .active_transactions    (),
        .error_count            (),
        .transaction_count      (),
        .dropped_count          (dropped_count),
        .refused_count          (refused_count)
    );

```

In a generated bridge, `mon_preset = "lite"` selects this wrapper for every monitored port.

11.7 Design Notes

- `cam_clear` keeps the `_mon` wrapper's name so the two are pin-compatible on the monitor side; the lite has a table, not a CAM.
 - `MAX_TRANSACTIONS` defaults to 8 here against 16 on `axil5_master_wr_mon`: size it to the port's real outstanding depth, since refused commands surface as orphans.
 - Consumers that relied on `block_ready` to bound the table get drop-and-count instead; the count is always reported, the identity of the lost events is not.
-

11.8 Related Modules

- `axil5_master_wr_mon` – the full-monitor sibling

- `axil5_master_wr` – the core this wraps
 - [axi_monitor_lite](#) – the monitor itself
-

11.9 Testing

`val/amba/monitor-lite/test_axil5_master_wr_monlite.py` exercises this module: the same TB and scenarios as `val/amba/test_axil5_master_wr_mon.py`, with the DUT swapped.

```
source env_python
make -C val/amba/monitor-lite run-axil5_master_wr_monlite-gate
```

Last Updated: 2026-09-26

11.10 Navigation

- [← Back to Monitor Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)

12 AXIL5 Slave Read

Module: `axil5_slave_rd.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd`

12.1 Overview

The AXIL5 Slave Read provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd` with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled

- Bit-identical to the AXI4-Lite module of the same name with all groups disabled

12.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	

Parameter	Type	Default	Description
PW	int	$(DW / 64) > 0$? $(DW / 64) :$ 1	
ARSize	int	$AW + 3 +$	
RSize	int	$DW + 2 +$	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

12.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_* side.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
s_axil_araddr	Input	$[AW-1:0]$	
s_axil_arprot	Input	$[2:0]$	
s_axil_arlock	Input	1	
s_axil_aruser	Input	$[UW-1:0]$	
s_axil_artrac e	Input	1	
s_axil_arloop	Input	$[LW-1:0]$	
s_axil_armpam	Input	$[MW-1:0]$	
s_axil_armeci d	Input	$[EW-1:0]$	
s_axil_arnsai d	Input	$[NW-1:0]$	
s_axil_arvali d	Input	1	
s_axil_arread y	Output	1	
fub_araddr	Output	$[AW-1:0]$	
fub_arprot	Output	$[2:0]$	

Port	Direction	Width	Description
fub_arlock	Output	1	
fub_aruser	Output	[UW-1:0]	
fub_artrace	Output	1	
fub_arloop	Output	[LW-1:0]	
fub_armpam	Output	[MW-1:0]	
fub_armecid	Output	[EW-1:0]	
fub_arnsaid	Output	[NW-1:0]	
fub_arvalid	Output	1	
fub_arready	Input	1	
fub_rdata	Input	[DW-1:0]	
fub_rresp	Input	[1:0]	
fub_ruser	Input	[UW-1:0]	
fub_rtrace	Input	1	
fub_rloop	Input	[LW-1:0]	
fub_rpoison	Input	[PW-1:0]	
fub_rvalid	Input	1	
fub_rready	Output	1	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rloop	Output	[LW-1:0]	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
busy	Output	1	

12.4 Functional Description

12.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

12.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH` = `AXIL_DATA_WIDTH` = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

12.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

12.5 Timing Characteristics

12.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

12.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

12.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AR        (2),
    .SKID_DEPTH_R         (4)
) u_axil5_slave_rd (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // s_axil_ar
    .s_axil_araddr       (s_axil_araddr),
    // ... `s_axil_arprot`, `s_axil_arlock`, `s_axil_aruser`, +7 more
    // fub_ar
    .fub_araddr         (fub_araddr),
    // ... `fub_arprot`, `fub_arlock`, `fub_aruser`, +7 more
    // fub_r
    .fub_rdata          (fub_rdata),
    // ... `fub_rresp`, `fub_ruser`, `fub_rtrace`, +4 more
    // s_axil_r
    .s_axil_rdata        (s_axil_rdata),
    // ... `s_axil_rresp`, `s_axil_ruser`, `s_axil_rtrace`, +4 more
    // status
    .busy               (busy)
);
```

12.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what `val/amba/test_axil5_axil4_equivalence.py` asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

12.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - `axil4_slave_rd` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

12.9 Testing

`val/amba/test_axil5_slave_rd.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

12.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

13 AXIL5 Slave Read, Clock-Gated

Module: axil5_slave_rd_cg.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_slave_rd_cg

13.1 Overview

The AXIL5 Slave Read, Clock-Gated provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_slave_rd_cg with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - One amba_clock_gate_ctrl gating the whole inner module
-

13.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	

Parameter	Type	Default	Description
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

13.3 Ports

The `s_axil_*` side faces the bus; your backend drives the `fub_*` side. The `cfg_cg_*` inputs run the clock gate.

Port	Direction	Width	Description
<code>aclk</code>	Input	1	
<code>aresetn</code>	Input	1	
<code>cfg_cg_enable</code>	Input	1	
<code>cfg_cg_idle_count</code>	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
<code>s_axil_araddr</code>	Input	[AW-1:0]	
<code>s_axil_arprot</code>	Input	[2:0]	
<code>s_axil_arlock</code>	Input	1	
<code>s_axil_aruser</code>	Input	[UW-1:0]	
<code>s_axil_artrace</code>	Input	1	
<code>s_axil_arloop</code>	Input	[LW-1:0]	
<code>s_axil_arpam</code>	Input	[MW-1:0]	
<code>s_axil_armecid</code>	Input	[EW-1:0]	
<code>s_axil_arnsaid</code>	Input	[NW-1:0]	
<code>s_axil_arvalid</code>	Input	1	
<code>s_axil_arready</code>	Output	1	
<code>fub_araddr</code>	Output	[AW-1:0]	
<code>fub_arprot</code>	Output	[2:0]	
<code>fub_arlock</code>	Output	1	
<code>fub_aruser</code>	Output	[UW-1:0]	
<code>fub_artrace</code>	Output	1	
<code>fub_arloop</code>	Output	[LW-1:0]	
<code>fub_arpam</code>	Output	[MW-1:0]	
<code>fub_armecid</code>	Output	[EW-1:0]	
<code>fub_arnsaid</code>	Output	[NW-1:0]	

Port	Direction	Width	Description
fub_arvalid	Output	1	
fub_arready	Input	1	
fub_rdata	Input	[DW-1:0]	
fub_rresp	Input	[1:0]	
fub_ruser	Input	[UW-1:0]	
fub_rtrace	Input	1	
fub_rloop	Input	[LW-1:0]	
fub_rpoison	Input	[PW-1:0]	
fub_rvalid	Input	1	
fub_rready	Output	1	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rloop	Output	[LW-1:0]	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

13.4 Functional Description

13.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

13.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

13.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it.

Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

13.5 Timing Characteristics

13.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

13.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

13.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd_cg #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH            (4),
```

```

        .LOOP_WIDTH           (3),
        .MPAM_WIDTH           (11),
        .MECID_WIDTH           (16),
        .NSAID_WIDTH           (4),
        .ENABLE_USER           (1),
        .ENABLE_TRACE           (1),
        .ENABLE_LOOP           (1),
        .ENABLE_MPAM           (1),
        .ENABLE_MECID           (1),
        .ENABLE_NSAID           (1),
        .ENABLE_POISON           (1),
        .ENABLE_LOCK           (1),
        .SKID_DEPTH_AR           (2),
        .SKID_DEPTH_R           (4),
        .CG_IDLE_COUNT_WIDTH (4)
    ) u_axil5_slave_rd_cg (
        // clock/reset
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // configuration
        .cfg_cg_enable           (cfg_cg_enable),
        .cfg_cg_idle_count       (cfg_cg_idle_count),
        // s_axil_ar
        .s_axil_araddr           (s_axil_araddr),
        // ... `s_axil_arprot`, `s_axil_arlock`, `s_axil_aruser`, +7 more
        // fub_ar
        .fub_araddr              (fub_araddr),
        // ... `fub_arprot`, `fub_arlock`, `fub_aruser`, +7 more
        // fub_r
        .fub_rdata                (fub_rdata),
        // ... `fub_rresp`, `fub_ruser`, `fub_rtrace`, +4 more
        // s_axil_r
        .s_axil_rdata             (s_axil_rdata),
        // ... `s_axil_rresp`, `s_axil_ruser`, `s_axil_rtrace`, +4 more
        // status
        .cg_gating                (cg_gating),
        .cg_idle                  (cg_idle)
    );

```

13.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The `*Size` parameters and the `AW/ DW` aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer’s READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating` – see `vault/handbook/design/clock-gating-activity-terms.md`.

busy is not brought out here. The plain `_cg` wrappers consume the base module’s busy internally as a wake term; use `cg_idle` or `cg_gating` for idle detection. The `_mon_cg` variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only. With it low the clock free-runs and the module behaves exactly like its base.

13.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - `axil4_slave_rd_cg` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

13.9 Testing

`val/amba/test_axil5_slave_rd_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to

mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

13.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

14 AXI5 Slave Read Monitor

Module: `axil5_slave_rd_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd_mon`

14.1 Overview

The AXI5 Slave Read Monitor provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd_mon` with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
-

14.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_slave_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the

monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_slave_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_slave_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_slave_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)

Parameter	Type	Default	Description
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Leave at 0 on AXI4-Lite. The wrapper hardwires cmd_id/data_id/response_id to 1'b0, so enabling this with ID_MATCH_BASE above 0 makes id_owned(0) false for every transaction and drops ALL monitoring.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE = 1. Must stay 0 on AXI4-Lite – every transaction reports ID 0.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes)

Parameter	Type	Default	Description
			everything).
ACTIVE_TRANS_THR ESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STA GE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOG IC	bit	1'b1	
ENABLE_TIMEOUT_L OGIC	bit	1'b1	
ENABLE_COMPL_LOG IC	bit	1'b1	
ENABLE_THRESHOLD _LOGIC	bit	1'b1	
ENABLE_PERF_LOGI C	bit	1'b1	
ENABLE_DEBUG_LOG IC	bit	1'b0	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	

Parameter	Type	Default	Description
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0014	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h02	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

14.3 Ports

The `s_axil_*` side faces the bus; your backend drives the `fub_axil_*` side. The `cfg_*` and `monbus_*` ports belong to the monitor.

Port	Direction	Width	Description
<code>aclk</code>	Input	1	
<code>aresetn</code>	Input	1	
<code>cam_clear</code>	Input	1	sync clear of the monitor trans CAM
<code>s_axil_araddr</code>	Input	[AW-1:0]	
<code>s_axil_arprot</code>	Input	[2:0]	
<code>s_axil_arlock</code>	Input	1	
<code>s_axil_aruser</code>	Input	[UW-1:0]	
<code>s_axil_artrace</code>	Input	1	
<code>s_axil_arloop</code>	Input	[LW-1:0]	
<code>s_axil_arpam</code>	Input	[MW-1:0]	
<code>s_axil_armecid</code>	Input	[EW-1:0]	
<code>s_axil_arnsaid</code>	Input	[NW-1:0]	
<code>s_axil_arvalid</code>	Input	1	
<code>s_axil_arready</code>	Output	1	
<code>s_axil_rdata</code>	Output	[DW-1:0]	
<code>s_axil_rresp</code>	Output	[1:0]	
<code>s_axil_ruser</code>	Output	[UW-1:0]	
<code>s_axil_rtrace</code>	Output	1	
<code>s_axil_rloop</code>	Output	[LW-1:0]	
<code>s_axil_rpoison</code>	Output	[PW-1:0]	
<code>s_axil_rvalid</code>	Output	1	
<code>s_axil_rready</code>	Input	1	
<code>fub_axil_araddr</code>	Output	[AW-1:0]	
<code>fub_axil_arprot</code>	Output	[2:0]	

Port	Direction	Width	Description
fub_axil_arlock	Output	1	
fub_axil_aruser	Output	[UW-1:0]	
fub_axil_artrace	Output	1	
fub_axil_arloop	Output	[LW-1:0]	
fub_axil_arpam	Output	[MW-1:0]	
fub_axil_armecid	Output	[EW-1:0]	
fub_axil_arnsaid	Output	[NW-1:0]	
fub_axil_arvalid	Output	1	
fub_axil_arready	Input	1	
fub_axil_rdata	Input	[DW-1:0]	
fub_axil_rresp	Input	[1:0]	
fub_axil_ruser	Input	[UW-1:0]	
fub_axil_rtrace	Input	1	
fub_axil_rloop	Input	[LW-1:0]	
fub_axil_rpoison	Input	[PW-1:0]	
fub_axil_rvalid	Input	1	
fub_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug

Port	Direction	Width	Description
cfg_timeout_cycles	Input	[15:0]	packets Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_en	Input	1	

Port	Direction	Width	Description
enable			
cfg_addr_range_enable	Input	$[(N_ADDR_RANGES > 0) ? N_ADDR_RANGES : 1] - 1:0]$	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	$[AW-1:0]$	
cfg_addr_filter_high	Input	$[AW-1:0]$	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	$[AW-1:0] \times N_ADDR_RANGES$	Low bound of each address-range comparator; only present when

Port	Direction	Width	Description
			N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	

Port	Direction	Width	Description
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

14.4 Functional Description

14.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

14.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

14.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

14.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

14.6 Timing Characteristics

14.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

14.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

14.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd_mon #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .ACLK_MHZ           (100),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES       (0),
    .UNIT_ID             (8'h02),
    .AGENT_ID            (16'h0014),
    .MAX_TRANSACTIONS    (8),
    .USE_WDATA_ORDER_Q   (1'b0),
```

```

.NUM_BANKS          (1),
.ID_FILTER_ENABLE    (1'b0),
.ADDR_FILTER_ENABLE  (1'b0),
.ID_MATCH_BASE       (0),
.ID_MATCH_COUNT      (0),
.ENABLE_FILTERING     (1),
.ADD_PIPELINE_STAGE  (0),
.ENABLE_ERROR_LOGIC   (1'b1),
.ENABLE_TIMEOUT_LOGIC(1'b1),
.ENABLE_COMPL_LOGIC   (1'b1),
.ENABLE_THRESHOLD_LOGIC(1'b1),
.ENABLE_PERF_LOGIC    (1'b1),
.ENABLE_DEBUG_LOGIC   (1'b0),
.USER_WIDTH          (4),
.LOOP_WIDTH           (3),
.MPAM_WIDTH           (11),
.MECID_WIDTH          (16),
.NSAID_WIDTH          (4),
.ENABLE_USER          (1),
.ENABLE_TRACE         (1),
.ENABLE_LOOP          (1),
.ENABLE_MPAM          (1),
.ENABLE_MECID         (1),
.ENABLE_NSAID         (1),
.ENABLE_POISON        (1),
.ENABLE_LOCK          (1)
) u_axil5_slave_rd_mon (
    // clock/reset
    .aclk              (aclk),
    .aresetn           (aresetn),
    // status
    .cam_clear         (cam_clear),
    // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+31 more
    // s_axil_ar
    .s_axil_araddr      (s_axil_araddr),
    // ... `s_axil_arprot`, `s_axil_arlock`, `s_axil_aruser`, +7 more
    // s_axil_r
    .s_axil_rdata       (s_axil_rdata),
    // ... `s_axil_rresp`, `s_axil_ruser`, `s_axil_rtrace`, +4 more
    // configuration
    .cfg_monitor_enable (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
    // monitor bus
    .monbus_valid       (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);

```

14.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_slave_rd_mon](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

14.9 Testing

`val/amba/test_axil5_slave_rd_mon.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

14.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

15 AXI5 Slave Read Monitor, Clock-Gated

Module: `axil5_slave_rd_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd_mon_cg`

15.1 Overview

The AXI5 Slave Read Monitor, Clock-Gated provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd_mon_cg` with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets
- Monitor plus clock gating; ready signals held low while gated

15.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon; outputs tied
N_ADDR_RANGES	int	0	0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding

Parameter	Type	Default	Description
			transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	

Parameter	Type	Default	Description
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0014	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h02	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes

every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until d6266344 removed those overrides.

15.3 Ports

The `s_axil_*` side faces the bus; your backend drives the `fub_axil_*` side. The `cfg_*` and `monbus_*` ports belong to the monitor, and `cfg_cg_*` runs the clock gate.

Port	Direction	Width	Description
<code>aclk</code>	Input	1	
<code>aresetn</code>	Input	1	
<code>cam_clear</code>	Input	1	sync clear of the monitor trans CAM
<code>s_axil_araddr</code>	Input	[AW-1:0]	
<code>s_axil_arprot</code>	Input	[2:0]	
<code>s_axil_arlock</code>	Input	1	
<code>s_axil_aruser</code>	Input	[UW-1:0]	
<code>s_axil_artrace</code>	Input	1	
<code>s_axil_arloop</code>	Input	[LW-1:0]	
<code>s_axil_arpam</code>	Input	[MW-1:0]	
<code>s_axil_armecid</code>	Input	[EW-1:0]	
<code>s_axil_arnsaid</code>	Input	[NW-1:0]	
<code>s_axil_arvalid</code>	Input	1	
<code>s_axil_arready</code>	Output	1	
<code>s_axil_rdata</code>	Output	[DW-1:0]	
<code>s_axil_rresp</code>	Output	[1:0]	
<code>s_axil_ruser</code>	Output	[UW-1:0]	
<code>s_axil_rtrace</code>	Output	1	
<code>s_axil_rloop</code>	Output	[LW-1:0]	
<code>s_axil_rpoison</code>	Output	[PW-1:0]	
<code>s_axil_rvalid</code>	Output	1	
<code>s_axil_rready</code>	Input	1	

Port	Direction	Width	Description
fub_axil_araddr	Output	[AW-1:0]	
fub_axil_arprot	Output	[2:0]	
fub_axil_arlock	Output	1	
fub_axil_aruser	Output	[UW-1:0]	
fub_axil_artrace	Output	1	
fub_axil_arloop	Output	[LW-1:0]	
fub_axil_armpam	Output	[MW-1:0]	
fub_axil_armecid	Output	[EW-1:0]	
fub_axil_arnsaid	Output	[NW-1:0]	
fub_axil_arvalid	Output	1	
fub_axil_arready	Input	1	
fub_axil_rdata	Input	[DW-1:0]	
fub_axil_rresp	Input	[1:0]	
fub_axil_ruser	Input	[UW-1:0]	
fub_axil_rtrace	Input	1	
fub_axil_rloop	Input	[LW-1:0]	
fub_axil_rpoison	Input	[PW-1:0]	
fub_axil_rvalid	Input	1	
fub_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets

Port	Direction	Width	Description
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask

Port	Direction	Width	Description
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1) - 1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor

Port	Direction	Width	Description
			stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	

Port	Direction	Width	Description
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

15.4 Functional Description

15.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

15.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

15.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

15.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1.

axi_monitor_trans_mgr fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets ADDR_FILTER_ENABLE but leaves cfg_addr_filter_enable low filters nothing and looks broken. The parameter and the runtime port are both required.

15.6 Timing Characteristics

15.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

15.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, ASize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

15.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd_mon_cg #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
```

```

.AXIL_ADDR_WIDTH      (32),
.AXIL_DATA_WIDTH      (32),
.USE_MONITOR          (1'b1),
.N_ADDR_RANGES        (0),
.ADDR_FILTER_ENABLE   (1'b0),
.ACLK_MHZ             (100),
.USE_WDATA_ORDER_Q    (1'b0),
.NUM_BANKS            (1),
.UNIT_ID              (8'h02),
.AGENT_ID              (16'h0014),
.MAX_TRANSACTIONS     (8),
.ENABLE_FILTERING     (1),
.ADD_PIPELINE_STAGE   (0),
.ENABLE_ERROR_LOGIC   (1'b1),
.ENABLE_TIMEOUT_LOGIC (1'b1),
.ENABLE_COMPL_LOGIC   (1'b1),
.ENABLE_THRESHOLD_LOGIC (1'b1),
.ENABLE_PERF_LOGIC    (1'b1),
.ENABLE_DEBUG_LOGIC   (1'b0),
.CG_IDLE_COUNT_WIDTH  (4),
.USER_WIDTH           (4),
.LOOP_WIDTH           (3),
.MPAM_WIDTH           (11),
.MECID_WIDTH          (16),
.NSAID_WIDTH          (4),
.ENABLE_USER          (1),
.ENABLE_TRACE         (1),
.ENABLE_LOOP          (1),
.ENABLE_MPAM          (1),
.ENABLE_MECID         (1),
.ENABLE_NSAID         (1),
.ENABLE_POISON        (1),
.ENABLE_LOCK          (1)
) u_axil5_slave_rd_mon_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+32 more
    // s_axil_ar
    .s_axil_araddr       (s_axil_araddr),
    // ... `s_axil_arprot`, `s_axil_arlock`, `s_axil_aruser`, +7 more
    // s_axil_r
    .s_axil_rdata        (s_axil_rdata),
    // ... `s_axil_rresp`, `s_axil_ruser`, `s_axil_rtrace`, +4 more
    // configuration

```

```

        .cfg_monitor_enable (cfg_monitor_enable),
        // ... `cfg_error_enable`, `cfg_timeout_enable`,
        `cfg_perf_enable`, +30 more
        // monitor bus
        .monbus_valid        (monbus_valid)
        // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
    );

```

15.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - axil4_slave_rd_mon_cg – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

15.9 Testing

val/amba/test_axil5_slave_rd_mon_cg.py drives this module with the AXI5-Lite BFM and **every optional group enabled** – TBClasses/axil5 sets the group widths in COMPONENT_KWARGS to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

15.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

16 AXI5-Lite Slave Read with Lite Monitor

Module: axil5_slave_rd_monlite.sv **Location:** rtl/amba/axil5/ **Status:** Production Ready (amba/monitor-lite TASK-001, 2026-09-26)

16.1 Overview

`axil5_slave_rd_monlite` is `axil5_slave_rd` with `axi_monitor_lite` watching its slave-side read channels. It is the lite-monitor sibling of `axil5_slave_rd_mon`: same core, same taps, same 128-bit `monitor_packet_t` on the same monbus with the same `UNIT_ID/AGENT_ID`, so the monbus arbiter, the AXI-Lite group, the tally and the host tooling cannot tell the two apart. The difference is what it costs and what it can be asked for: about a fifth of the monitor LUTs (677 against 3,249 per read monitor in the same bridge fixture, see [axi_monitor_lite](#)), and only the four packet classes the lite emits: error, timeout, completion and active-count threshold.

16.1.1 What is not here

Compared with `axil5_slave_rd_mon`: no performance window or counters, no debug packets, no address-range checker, no ID or address filters, no per-event masks, no latency threshold, and no `block_ready`. The lite never stalls the port: an event it cannot deliver is dropped and counted (`dropped_count`, reported as an `EVENT_DROPPED` packet), and a command that finds no free table entry is counted (`refused_count`) and left untracked, so its beats report as orphans.

16.1.2 Key Features

- Every `axil5_slave_rd` parameter and port, declared verbatim and passed through by name
 - `axi_monitor_lite` on the s_-side taps; `USE_MONITOR=0` omits it and ties its outputs
 - Nine config inputs: the enables, the microsecond timeout, the tick LUT index and the packet-type mask
 - Five status outputs, two of them (`dropped_count`, `refused_count`) unique to the lite
-

16.2 Parameters

16.2.1 Monitor Parameters

Parameter	Type	Default	Description
<code>USE_MONITOR</code>	bit	<code>1'b1</code>	0 = omit the monitor, tie its outputs
<code>UNIT_ID</code>	logic [7:0]	<code>8'h02</code>	Unit id in every packet

Parameter	Type	Default	Description
AGENT_ID	logic [15:0]	16'h0014	Agent id in every packet
MAX_TRANSACTIONS	int	8	table entries; a command finding none is counted, not tracked
ACTIVE_TRANS_THR ESHOLD	int	MAX_TRANSACTIONS / 2	Table occupancy that fires a threshold packet (rising edge)
OUT_DEPTH	int	4	monbus output queue, a power of two
ACLK_MHZ	int	100	Clock in MHz; keeps the 1 us tick exact
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT lower bound (cfg_freq_sel indexes it)
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT upper bound

16.2.2 Core Parameters (passed through to axil5_slave_rd)

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	as on axil5_slave_rd
AXIL_DATA_WIDTH	int	32	as on axil5_slave_rd
USER_WIDTH	int	4	as on axil5_slave_rd
LOOP_WIDTH	int	3	as on axil5_slave_rd

Parameter	Type	Default	Description
MPAM_WIDTH	int	11	as on axil5_slave_rd
MECID_WIDTH	int	16	as on axil5_slave_rd
NSAID_WIDTH	int	4	as on axil5_slave_rd
ENABLE_USER	bit	1	as on axil5_slave_rd
ENABLE_TRACE	bit	1	as on axil5_slave_rd
ENABLE_LOOP	bit	1	as on axil5_slave_rd
ENABLE_MPAM	bit	1	as on axil5_slave_rd
ENABLE_MECID	bit	1	as on axil5_slave_rd
ENABLE_NSAID	bit	1	as on axil5_slave_rd
ENABLE_POISON	bit	1	as on axil5_slave_rd
ENABLE_LOCK	bit	1	as on axil5_slave_rd
SKID_DEPTH_AR	int	2	as on axil5_slave_rd
SKID_DEPTH_R	int	4	as on axil5_slave_rd
AW	int	AXIL_ADDR_WIDTH	as on axil5_slave_rd
DW	int	AXIL_DATA_WIDTH	as on axil5_slave_rd
UW	int	USER_WIDTH	as on axil5_slave_rd
LW	int	LOOP_WIDTH	as on axil5_slave_rd
MW	int	MPAM_WIDTH	as on axil5_slave_rd
EW	int	MECID_WIDTH	as on

Parameter	Type	Default	Description
NW	int	NSAID_WIDTH	axil5_slave_rd as on axil5_slave_rd
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	as on axil5_slave_rd
ARSize	int	AW + 3 +	as on axil5_slave_rd
RSize	int	DW + 2 +	as on axil5_slave_rd

16.3 Ports

16.3.1 Monitor Control, Bus and Status

Port	Direction	Type	Description
cam_clear	input	logic	sync clear of the monitor's table (named as on the _mon wrapper: pin-compatible)
cfg_monitor_enable	input	logic	0 = monitor inert, table held clear
cfg_error_enable	input	logic	
cfg_timeout_enable	input	logic	
cfg_compl_enable	input	logic	
cfg_threshold_enable	input	logic	
cfg_timeout_cycles	input	logic [15:0]	MICROSECONDS of no progress; 0 = never
cfg_freq_sel	input	logic [3:0]	counter_freq_invariant LUT index
cfg_axi_pkt_mask	input	logic [15:0]	drop mask by packet type

Port	Direction	Type	Description
i_mon_time	input	monitor_common_pkg::monbus_times_tamp_t	
monbus_valid	output	logic	
monbus_ready	input	logic	
monbus_packet	output	monitor_common_pkg::monitor_packet_t	
monbus_timestamp	output	monitor_common_pkg::monbus_times_tamp_t	
active_transactions	output	logic [7:0]	entries in the table
error_count	output	logic [15:0]	error + timeout packets emitted
transaction_count	output	logic [31:0]	completion packets emitted
dropped_count	output	logic [15:0]	events lost to monbus backpressure since the last report
refused_count	output	logic [15:0]	commands that found no free table entry

16.3.2 Core Ports (as on axil5_slave_rd)

Port	Direction	Type	Description
aclk	input	logic	
aresetn	input	logic	
s_axil_araddr	input	logic [AW-1:0]	
s_axil_arprot	input	logic [2:0]	
s_axil_arlock	input	logic	
s_axil_aruser	input	logic [UW-1:0]	

Port	Direction	Type	Description
s_axil_artrac e	input	logic	
s_axil_arloop	input	logic [LW- 1:0]	
s_axil_arpam	input	logic [MW- 1:0]	
s_axil_armeci d	input	logic [EW- 1:0]	
s_axil_arnsai d	input	logic [NW- 1:0]	
s_axil_arvali d	input	logic	
s_axil_arread y	output	logic	
fub_axil_arad dr	output	logic [AW- 1:0]	
fub_axil_arpr ot	output	logic [2:0]	
fub_axil_arlo ck	output	logic	
fub_axil_arus er	output	logic [UW- 1:0]	
fub_axil_artr ace	output	logic	
fub_axil_arlo op	output	logic [LW- 1:0]	
fub_axil_arp am	output	logic [MW- 1:0]	
fub_axil_arme cid	output	logic [EW- 1:0]	
fub_axil_arns aid	output	logic [NW- 1:0]	
fub_axil_arva lid	output	logic	
fub_axil_arre ady	input	logic	
fub_axil_rdat a	input	logic [DW- 1:0]	
fub_axil_rres p	input	logic [1:0]	
fub_axil_ruse	input	logic [UW-	

Port	Direction	Type	Description
r		1:0]	
fub_axil_rtra	input	logic	
ce			
fub_axil_rloop	input	logic [LW-1:0]	
p			
fub_axil_rpoison	input	logic [PW-1:0]	
fub_axil_rvalid	input	logic	
id			
fub_axil_rready	output	logic	
dy			
s_axil_rdata	output	logic [DW-1:0]	
s_axil_rresp	output	logic [1:0]	
s_axil_ruser	output	logic [UW-1:0]	
s_axil_rtrace	output	logic	
s_axil_rloop	output	logic [LW-1:0]	
s_axil_rpoison	output	logic [PW-1:0]	
s_axil_rvalid	output	logic	
s_axil_rready	input	logic	
busy	output	logic	

16.4 Functional Description

The core is instantiated untouched; unlike `axil5_slave_rd_mon` there is no gating of the command handshake, because the lite has no admission stall. The monitor taps the `s_`-side command, data and response channels through three valids gated by `cfg_monitor_enable`, which also holds the table clear while low. `cfg_timeout_cycles` is a microsecond count of no progress on an entry, passed at full width; 0 means never. The active-count threshold is the parameter `ACTIVE_TRANS_THRESHOLD`, as on the `_mon` wrapper.

The lite's own behaviour – per-ID linked-list attribution, stamps instead of counters, the registered event stage, the 4-entry output queue and drop-and-count – is on the [axi_monitor_lite](#) page.

16.5 Timing Characteristics

The datapath timing is axil5_slave_rd's. The monitor adds no gating to any handshake and no path into the datapath; its own paths are two short cycles (attribution, then packet formatting) and met 10 ns on an Artix-7 100T -1 with margin inside bridge_1x2_rd_lite_mon.

16.6 Usage Examples

```
axil5_slave_rd_monlite #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .UNIT_ID              (8'h02),
    .AGENT_ID              (16'h0014),
    .MAX_TRANSACTIONS      (8)
) u_axil5_slave_rd_monlite (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    // ... the axil5_slave_rd ports, by name ...
    .cam_clear             (1'b0),
    .cfg_monitor_enable    (1'b1),
    .cfg_error_enable      (1'b1),
    .cfg_timeout_enable    (1'b1),
    .cfg_compl_enable      (1'b0),
    .cfg_threshold_enable  (1'b0),
    .cfg_timeout_cycles    (16'd1000),
    .cfg_freq_sel          (4'd0),
    .cfg_axi_pkt_mask      (16'h0000),
    .i_mon_time            (mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .active_transactions   (),
    .error_count           (),
    .transaction_count     (),
    .dropped_count         (dropped_count),
    .refused_count         (refused_count)
);
```

In a generated bridge, mon_preset = "lite" selects this wrapper for every monitored port.

16.7 Design Notes

- `cam_clear` keeps the `_mon` wrapper's name so the two are pin-compatible on the monitor side; the lite has a table, not a CAM.
 - `MAX_TRANSACTION`s defaults to 8 here against 16 on `axil5_slave_rd_mon`: size it to the port's real outstanding depth, since refused commands surface as orphans.
 - Consumers that relied on `block_ready` to bound the table get drop-and-count instead; the count is always reported, the identity of the lost events is not.
-

16.8 Related Modules

- `axil5_slave_rd_mon` – the full-monitor sibling
 - `axil5_slave_rd` – the core this wraps
 - [axi_monitor_lite](#) – the monitor itself
-

16.9 Testing

`val/amba/monitor-lite/test_axil5_slave_rd_monlite.py` exercises this module: the same TB and scenarios as `val/amba/test_axil5_slave_rd_mon.py`, with the DUT swapped.

```
source env_python
make -C val/amba/monitor-lite run-axil5_slave_rd_monlite-gate
```

Last Updated: 2026-09-26

16.10 Navigation

- [← Back to Monitor Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)

17 AXIL5 Slave Write

Module: axil5_slave_wr.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_slave_wr

17.1 Overview

The AXIL5 Slave Write provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_slave_wr with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
-

17.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	

Parameter	Type	Default	Description
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AWSize	int	AW + 3 +	
WSize	int	DW + SW +	
BSize	int	2 +	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

17.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_* side.

Port	Direction	Width	Description
ack	Input	1	

Port	Direction	Width	Description
aresetn	Input	1	
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrac e	Input	1	
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awmeci d	Input	[EW-1:0]	
s_axil_awnsai d	Input	[NW-1:0]	
s_axil_awvali d	Input	1	
s_axil_awread y	Output	1	
fub_awaddr	Output	[AW-1:0]	
fub_awprot	Output	[2:0]	
fub_awlock	Output	1	
fub_awuser	Output	[UW-1:0]	
fub_awtrace	Output	1	
fub_awloop	Output	[LW-1:0]	
fub_awmpam	Output	[MW-1:0]	
fub_awmecid	Output	[EW-1:0]	
fub_awnsaid	Output	[NW-1:0]	
fub_awvalid	Output	1	
fub_awready	Input	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[SW-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoiso n	Input	[PW-1:0]	

Port	Direction	Width	Description
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
fub_wdata	Output	[DW-1:0]	
fub_wstrb	Output	[SW-1:0]	
fub_wuser	Output	[UW-1:0]	
fub_wpoison	Output	[PW-1:0]	
fub_wvalid	Output	1	
fub_wready	Input	1	
fub_bresp	Input	[1:0]	
fub_buser	Input	[UW-1:0]	
fub_btrace	Input	1	
fub_bloop	Input	[LW-1:0]	
fub_bvalid	Input	1	
fub_bready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
busy	Output	1	

17.4 Functional Description

17.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER,	USER_WIDTH

Group	Parameter	Signals on this module	Width
		BUSER, RUSER	
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

17.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

17.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it.

Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

17.5 Timing Characteristics

17.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

17.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

17.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```

axil5_slave_wr #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AW        (2),
    .SKID_DEPTH_W         (4),
    .SKID_DEPTH_B         (2)
) u_axil5_slave_wr (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // s_axil_aw
    .s_axil_awaddr        (s_axil_awaddr),
    // ... `s_axil_awprot`, `s_axil_awlock`, `s_axil_awuser`, +7 more
    // fub_aw
    .fub_awaddr           (fub_awaddr),
    // ... `fub_awprot`, `fub_awlock`, `fub_awuser`, +7 more
    // s_axil_w
    .s_axil_wdata          (s_axil_wdata),
    // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
    // fub_w
    .fub_wdata            (fub_wdata),
    // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
    // fub_b
    .fub_bresp            (fub_bresp),
    // ... `fub_buser`, `fub_btrace`, `fub_bloop`, +2 more
    // s_axil_b
    .s_axil_bresp          (s_axil_bresp),
    // ... `s_axil_buser`, `s_axil_btrace`, `s_axil_bloop`, +2 more
    // status
    .busy                  (busy)
);

```

17.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what `val/amba/test_axil5_axil4_equivalence.py` asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

17.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - `axil4_slave_wr` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

17.9 Testing

`val/amba/test_axil5_slave_wr.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

17.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

18 AXIL5 Slave Write, Clock-Gated

Module: axil5_slave_wr_cg.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_slave_wr_cg

18.1 Overview

The AXIL5 Slave Write, Clock-Gated provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_slave_wr_cg with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - One amba_clock_gate_ctrl gating the whole inner module
-

18.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	

Parameter	Type	Default	Description
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

18.3 Ports

The `s_axil_*` side faces the bus; your backend drives the `fub_*` side. The `cfg_cg_*` inputs run the clock gate.

Port	Direction	Width	Description
<code>aclk</code>	Input	1	
<code>aresetn</code>	Input	1	
<code>cfg_cg_enable</code>	Input	1	
<code>cfg_cg_idle_count</code>	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
<code>s_axil_awaddr</code>	Input	[AW-1:0]	
<code>s_axil_awprot</code>	Input	[2:0]	
<code>s_axil_awlock</code>	Input	1	
<code>s_axil_awuser</code>	Input	[UW-1:0]	
<code>s_axil_awtrace</code>	Input	1	
<code>s_axil_awloop</code>	Input	[LW-1:0]	
<code>s_axil_awmpam</code>	Input	[MW-1:0]	
<code>s_axil_awmeci</code>	Input	[EW-1:0]	
<code>s_axil_awnsaid</code>	Input	[NW-1:0]	
<code>s_axil_awvalid</code>	Input	1	
<code>s_axil_awready</code>	Output	1	
<code>fub_awaddr</code>	Output	[AW-1:0]	
<code>fub_awprot</code>	Output	[2:0]	
<code>fub_awlock</code>	Output	1	
<code>fub_awuser</code>	Output	[UW-1:0]	
<code>fub_awtrace</code>	Output	1	
<code>fub_awloop</code>	Output	[LW-1:0]	
<code>fub_awmpam</code>	Output	[MW-1:0]	
<code>fub_awmeci</code>	Output	[EW-1:0]	
<code>fub_awnsaid</code>	Output	[NW-1:0]	

Port	Direction	Width	Description
fub_awvalid	Output	1	
fub_awready	Input	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[SW-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
fub_wdata	Output	[DW-1:0]	
fub_wstrb	Output	[SW-1:0]	
fub_wuser	Output	[UW-1:0]	
fub_wpoison	Output	[PW-1:0]	
fub_wvalid	Output	1	
fub_wready	Input	1	
fub_bresp	Input	[1:0]	
fub_buser	Input	[UW-1:0]	
fub_btrace	Input	1	
fub_bloop	Input	[LW-1:0]	
fub_bvalid	Input	1	
fub_bready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty

Port	Direction	Width	Description
			indicator

18.4 Functional Description

18.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

18.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

18.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

18.5 Timing Characteristics

18.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the un stalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

18.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

18.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr_cg #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AW        (2),
    .SKID_DEPTH_W         (4),
    .SKID_DEPTH_B         (2),
    .CG_IDLE_COUNT_WIDTH  (4)
) u_axil5_slave_wr_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // configuration
    .cfg_cg_enable        (cfg_cg_enable),
    .cfg_cg_idle_count    (cfg_cg_idle_count),
    // s_axil_aw
    .s_axil_awaddr        (s_axil_awaddr),
    // ... `s_axil_awprot`, `s_axil_awlock`, `s_axil_awuser`, +7 more
    // fub_aw
    .fub_awaddr           (fub_awaddr),
    // ... `fub_awprot`, `fub_awlock`, `fub_awuser`, +7 more
```

```

// s_axil_w
.s_axil_wdata      (s_axil_wdata),
// ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
// fub_w
.fub_wdata         (fub_wdata),
// ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
// fub_b
.fub_bresp         (fub_bresp),
// ... `fub_buser`, `fub_btrace`, `fub_bloop`, +2 more
// s_axil_b
.s_axil_bresp      (s_axil_bresp),
// ... `s_axil_buser`, `s_axil_btrace`, `s_axil_bloop`, +2 more
// status
.cg_gating         (cg_gating),
.cg_idle           (cg_idle)
);

```

18.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFMs against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer’s READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into user_valid pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. val/amba/test_cg_peer_ready.py parks the peer READY high, holds every VALID low, and requires cg_gating – see vault/handbook/design/clock-gating-activity-terms.md.

busy is not brought out here. The plain `_cg` wrappers consume the base module's `busy` internally as a wake term; use `cg_idle` or `cg_gating` for idle detection. The `_mon_cg` variants do expose `busy`.

cfg_cg_enable is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only. With it low the clock free-runs and the module behaves exactly like its base.

18.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - `axil4_slave_wr_cg` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

18.9 Testing

`val/amba/test_axil5_slave_wr_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

18.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

19 AXIL5 Slave Write Monitor

Module: `axil5_slave_wr_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr_mon`

19.1 Overview

The AXIL5 Slave Write Monitor gives a slave device a buffered AXI5-Lite write interface — and keeps a transaction monitor on the path while it's at it.

If you take one thing from this page, take this: AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr_mon` with those groups threaded through the packed SKID payload. Know the AXI4-Lite part and you know this one.

19.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
-

19.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_slave_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_slave_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_slave_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

A lite-monitor version of this wrapper exists as `axil5_slave_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	2	
SKID_DEPTH_B	int	2	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Leave at 0 on AXI4-Lite. The wrapper hardwires <code>cmd_id/data_id/re</code>

Parameter	Type	Default	Description
			sp_id to 1'b0, so enabling this with ID_MATCH_BASE above 0 makes id_owned(0) false for every transaction and drops ALL monitoring.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE = 1. Must stay 0 on AXI4-Lite – every transaction reports ID 0.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	

Parameter	Type	Default	Description
ENABLE_THRESHOLD _LOGIC	bit	1'b1	
ENABLE_PERF_LOGI C	bit	1'b1	
ENABLE_DEBUG_LOG IC	bit	1'b0	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0015	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with

Parameter	Type	Default	Description
UNIT_ID		8'h02	UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets) Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Force a *Size to a value the RTL didn't derive and every optional-group field turns into a part-select past the end of the vector — which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

19.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	

Port	Direction	Width	Description
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awmecid	Input	[EW-1:0]	
s_axil_awnsaid	Input	[NW-1:0]	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[DW/8-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
fub_axil_awaddr	Output	[AW-1:0]	
fub_axil_awprot	Output	[2:0]	
fub_axil_awlock	Output	1	
fub_axil_awuser	Output	[UW-1:0]	
fub_axil_awtrace	Output	1	
fub_axil_awloop	Output	[LW-1:0]	
fub_axil_awmpam	Output	[MW-1:0]	
fub_axil_awmecid	Output	[EW-1:0]	
fub_axil_awnsaid	Output	[NW-1:0]	

Port	Direction	Width	Description
fub_axil_awvalid	Output	1	
fub_axil_awready	Input	1	
fub_axil_wdata	Output	[DW-1:0]	
fub_axil_wstrb	Output	[DW/8-1:0]	
fub_axil_wuser	Output	[UW-1:0]	
fub_axil_wpoison	Output	[PW-1:0]	
fub_axil_wvalid	Output	1	
fub_axil_wready	Input	1	
fub_axil_bresp	Input	[1:0]	
fub_axil_buser	Input	[UW-1:0]	
fub_axil_btrace	Input	1	
fub_axil_bloop	Input	[LW-1:0]	
fub_axil_bvalid	Input	1	
fub_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in

Port	Direction	Width	Description
			MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_inva riant LUT index
cfg_latency_thre shold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_sele ct	Input	[15:0]	Error select for packet types
cfg_axi_error_ma sk	Input	[15:0]	Individual error event mask
cfg_axi_timeout_ mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_ma sk	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_m ask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mas k	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mas k	Input	[15:0]	Individual address match event mask
cfg_axi_debug_ma sk	Input	[15:0]	Individual debug event mask
cfg_addr_check_e nable	Input	1	
cfg_addr_range_e nable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 0)]	

Port	Direction	Width	Description
		1) - 1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x	High bound of

Port	Direction	Width	Description
		N_ADDR_RANGES	each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

19.4 Functional Description

AXI5-Lite adds eight optional signal groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when it's enabled — turn it off and those bits simply aren't in the vector.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

19.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH` = `AXIL_DATA_WIDTH` = 32 and the default group widths.)

That equivalence isn't academic. It's what `val/amba/test_axi5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

19.4.2 It transports; it does not interpret

This module is a pipe, not a policy engine. MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

One kindness to integrators: a disabled group's OUTPUT is driven to zero rather than left dangling, so if you disable a group downstream of a block that enables it, you see a defined value instead of X.

19.5 Design Notes

A handful of traps specific to the monitored variants. Each one is written down because someone hit it.

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It doesn't check MPAM/MECID/NSAID consistency, and it doesn't validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong — silently. That's the worst kind of wrong.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination. At least that one fails loud.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

19.6 Timing Characteristics

19.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the

Parameter	Default	Channel
		AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

19.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

19.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr_mon #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (2),
    .SKID_DEPTH_B       (2),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .ACLK_MHZ           (100),
    .USE_MONITOR        (1'b1),
    .N_ADDR_RANGES      (0),
    .UNIT_ID             (8'h02),
    .AGENT_ID           (16'h0015),
    .MAX_TRANSACTIONS   (8),
    .USE_WDATA_ORDER_Q  (1'b0),
    .NUM_BANKS          (1),
    .ID_FILTER_ENABLE   (1'b0),
```

```

.ADDR_FILTER_ENABLE (1'b0),
.ID_MATCH_BASE      (0),
.ID_MATCH_COUNT     (0),
.ENABLE_FILTERING   (1),
.ADD_PIPELINE_STAGE (0),
.ENABLE_ERROR_LOGIC (1'b1),
.ENABLE_TIMEOUT_LOGIC(1'b1),
.ENABLE_COMPL_LOGIC (1'b1),
.ENABLE_THRESHOLD_LOGIC(1'b1),
.ENABLE_PERF_LOGIC  (1'b1),
.ENABLE_DEBUG_LOGIC (1'b0),
.USER_WIDTH         (4),
.LOOP_WIDTH         (3),
.MPAM_WIDTH         (11),
.MECID_WIDTH        (16),
.NSAID_WIDTH        (4),
.ENABLE_USER        (1),
.ENABLE_TRACE       (1),
.ENABLE_LOOP        (1),
.ENABLE_MPAM        (1),
.ENABLE_MECID       (1),
.ENABLE_NSAID       (1),
.ENABLE_POISON      (1),
.ENABLE_LOCK        (1)
) u_axil5_slave_wr_mon (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+35 more
    // s_axil_aw
    .s_axil_awaddr       (s_axil_awaddr),
    // ... `s_axil_awprot`, `s_axil_awlock`, `s_axil_awuser`, +7 more
    // s_axil_w
    .s_axil_wdata        (s_axil_wdata),
    // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
    // s_axil_b
    .s_axil_bresp        (s_axil_bresp),
    // ... `s_axil_buser`, `s_axil_btrace`, `s_axil_bloop`, +2 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
    // monitor bus
    .monbus_valid        (monbus_valid)

```

```
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`  
);
```

19.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_slave_wr_mon](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

19.9 Testing

`val/amba/test_axil5_slave_wr_mon.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** — `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake — and the version you'd rather get.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically. That's exactly how you want a variant family to work.

19.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

20 AXI5 Slave Write Monitor, Clock-Gated

Module: `axil5_slave_wr_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr_mon_cg`

20.1 Overview

The AXI5 Slave Write Monitor, Clock-Gated gives a slave device a buffered AXI5-Lite write interface — the same monitor as the plain variant, with clock gating stacked on top and the ready signals held low while gated.

If you take one thing from this page, take this: AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr_mon_cg` with those groups threaded through the packed SKID payload. Know the AXI4-Lite part and you know this one.

20.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets
- Monitor plus clock gating; ready signals held low while gated

20.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	2	
SKID_DEPTH_B	int	2	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon; outputs tied
N_ADDR_RANGES	int	0	0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	

Parameter	Type	Default	Description
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	

Parameter	Type	Default	Description
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0015	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h02	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets

Parameter	Type	Default	Description
			cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Force a *Size to a value the RTL didn't derive and every optional-group field turns into a part-select past the end of the vector — which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

20.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awmecid	Input	[EW-1:0]	
s_axil_awnsaid	Input	[NW-1:0]	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[DW/8-1:0]	
s_axil_wuser	Input	[UW-1:0]	

Port	Direction	Width	Description
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
fub_axil_awaddr	Output	[AW-1:0]	
fub_axil_awprot	Output	[2:0]	
fub_axil_awlock	Output	1	
fub_axil_awuser	Output	[UW-1:0]	
fub_axil_awtrace	Output	1	
fub_axil_awloop	Output	[LW-1:0]	
fub_axil_awmpam	Output	[MW-1:0]	
fub_axil_awmecid	Output	[EW-1:0]	
fub_axil_awnsaid	Output	[NW-1:0]	
fub_axil_awvalid	Output	1	
fub_axil_awready	Input	1	
fub_axil_wdata	Output	[DW-1:0]	
fub_axil_wstrb	Output	[DW/8-1:0]	
fub_axil_wuser	Output	[UW-1:0]	
fub_axil_wpoison	Output	[PW-1:0]	
fub_axil_wvalid	Output	1	
fub_axil_wready	Input	1	
fub_axil_bresp	Input	[1:0]	
fub_axil_buser	Input	[UW-1:0]	
fub_axil_btrace	Input	1	
fub_axil_bloop	Input	[LW-1:0]	

Port	Direction	Width	Description
fub_axil_bvalid	Input	1	
fub_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask

Port	Direction	Width	Description
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t	The monitor

Port	Direction	Width	Description
		(128)	packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count

Port	Direction	Width	Description
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

20.4 Functional Description

AXI5-Lite adds eight optional signal groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when it's enabled — turn it off and those bits simply aren't in the vector.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

20.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence isn't academic. It's what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

20.4.2 It transports; it does not interpret

This module is a pipe, not a policy engine. MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried

with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

One kindness to integrators: a disabled group's OUTPUT is driven to zero rather than left dangling, so if you disable a group downstream of a block that enables it, you see a defined value instead of X.

20.5 Design Notes

A handful of traps specific to the monitored variants. Each one is written down because someone hit it.

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It doesn't check MPAM/MECID/NSAID consistency, and it doesn't validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong — silently. That's the worst kind of wrong.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination. At least that one fails loud.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

20.6 Timing Characteristics

20.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

20.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

20.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr_mon_cg #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (2),
    .SKID_DEPTH_B       (2),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .USE_MONITOR        (1'b1),
    .N_ADDR_RANGES      (0),
    .ADDR_FILTER_ENABLE (1'b0),
    .ACLK_MHZ           (100),
    .USE_WDATA_ORDER_Q  (1'b0),
    .NUM_BANKS          (1),
    .UNIT_ID            (8'h02),
    .AGENT_ID           (16'h0015),
    .MAX_TRANSACTIONS   (8),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .CG_IDLE_COUNT_WIDTH (4),
```

```

        .USER_WIDTH           (4),
        .LOOP_WIDTH           (3),
        .MPAM_WIDTH            (11),
        .MECID_WIDTH           (16),
        .NSAID_WIDTH           (4),
        .ENABLE_USER           (1),
        .ENABLE_TRACE           (1),
        .ENABLE_LOOP           (1),
        .ENABLE_MPAM           (1),
        .ENABLE_MECID           (1),
        .ENABLE_NSAID           (1),
        .ENABLE_POISON          (1),
        .ENABLE_LOCK           (1)
    ) u_axil5_slave_wr_mon_cg (
        // clock/reset
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // status
        .cam_clear               (cam_clear),
        // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+36 more
        // s_axil_aw
        .s_axil_awaddr           (s_axil_awaddr),
        // ... `s_axil_awprot`, `s_axil_awlock`, `s_axil_awuser`, +7 more
        // s_axil_w
        .s_axil_wdata            (s_axil_wdata),
        // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
        // s_axil_b
        .s_axil_bresp            (s_axil_bresp),
        // ... `s_axil_buser`, `s_axil_btrace`, `s_axil_bloop`, +2 more
        // configuration
        .cfg_monitor_enable      (cfg_monitor_enable),
        // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +30 more
        // monitor bus
        .monbus_valid            (monbus_valid)
        // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
    );

```

20.8 Related Modules

- [axil5_master_rd](#)
- [axil5_master_rd_cg](#)
- [axil5_master_rd_mon](#)
- [axil5_master_rd_mon_cg](#)

- [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_slave_wr_mon_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

20.9 Testing

`val/amba/test_axil5_slave_wr_mon_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** — `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake — and the version you'd rather get.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically. That's exactly how you want a variant family to work.

20.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

21 AXI5-Lite Slave Write with Lite Monitor

Module: `axil5_slave_wr_monlite.sv` **Location:** `rtl/amba/axil5/` **Status:** Production Ready
(`amba/monitor-lite TASK-001`, 2026-09-26)

21.1 Overview

`axil5_slave_wr_monlite` is `axil5_slave_wr` with `axi_monitor_lite` watching its slave-side write channels. It is the lite-monitor sibling of `axil5_slave_wr_mon`: same core, same taps, same 128-bit `monitor_packet_t` on the same monbus with the same `UNIT_ID/AGENT_ID`, so the monbus arbiter, the AXI-Lite group, the tally and the host tooling cannot tell the two apart. The difference is what it costs and what it can be asked for: about a fifth of the monitor LUTs (677 against 3,249 per read monitor in the same bridge fixture, see [axi_monitor_lite](#)), and only the four packet classes the lite emits: error, timeout, completion and active-count threshold.

21.1.1 What is not here

Compared with axil5_slave_wr_mon: no performance window or counters, no debug packets, no address-range checker, no ID or address filters, no per-event masks, no latency threshold, and no block_ready. The lite never stalls the port: an event it cannot deliver is dropped and counted (dropped_count, reported as an EVENT_DROPPED packet), and a command that finds no free table entry is counted (refused_count) and left untracked, so its beats report as orphans.

21.1.2 Key Features

- Every axil5_slave_wr parameter and port, declared verbatim and passed through by name
 - axi_monitor_lite on the s_-side taps; USE_MONITOR=0 omits it and ties its outputs
 - Nine config inputs: the enables, the microsecond timeout, the tick LUT index and the packet-type mask
 - Five status outputs, two of them (dropped_count, refused_count) unique to the lite
-

21.2 Parameters

21.2.1 Monitor Parameters

Parameter	Type	Default	Description
USE_MONITOR	bit	1'b1	0 = omit the monitor, tie its outputs
UNIT_ID	logic [7:0]	8'h02	Unit id in every packet
AGENT_ID	logic [15:0]	16'h0015	Agent id in every packet
MAX_TRANSACTIONS	int	8	table entries; a command finding none is counted, not tracked
ACTIVE_TRANS_THR ESHOLD	int	MAX_TRANSACTIONS / 2	Table occupancy that fires a threshold packet (rising edge)

Parameter	Type	Default	Description
OUT_DEPTH	int	4	monbus output queue, a power of two
ACLK_MHZ	int	100	Clock in MHz; keeps the 1 us tick exact
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT lower bound (cfg_freq_sel indexes it)
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Frequency-invariant tick LUT upper bound

21.2.2 Core Parameters (passed through to axil5_slave_wr)

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	as on axil5_slave_wr
AXIL_DATA_WIDTH	int	32	as on axil5_slave_wr
USER_WIDTH	int	4	as on axil5_slave_wr
LOOP_WIDTH	int	3	as on axil5_slave_wr
MPAM_WIDTH	int	11	as on axil5_slave_wr
MECID_WIDTH	int	16	as on axil5_slave_wr
NSAID_WIDTH	int	4	as on axil5_slave_wr
ENABLE_USER	bit	1	as on axil5_slave_wr
ENABLE_TRACE	bit	1	as on axil5_slave_wr
ENABLE_LOOP	bit	1	as on

Parameter	Type	Default	Description
ENABLE_MPAM	bit	1	axil5_slave_wr as on axil5_slave_wr
ENABLE_MECID	bit	1	as on axil5_slave_wr
ENABLE_NSAID	bit	1	as on axil5_slave_wr
ENABLE_POISON	bit	1	as on axil5_slave_wr
ENABLE_LOCK	bit	1	as on axil5_slave_wr
SKID_DEPTH_AW	int	2	as on axil5_slave_wr
SKID_DEPTH_W	int	4	as on axil5_slave_wr
SKID_DEPTH_B	int	2	as on axil5_slave_wr
AW	int	AXIL_ADDR_WIDTH	as on axil5_slave_wr
DW	int	AXIL_DATA_WIDTH	as on axil5_slave_wr
SW	int	DW / 8	as on axil5_slave_wr
UW	int	USER_WIDTH	as on axil5_slave_wr
LW	int	LOOP_WIDTH	as on axil5_slave_wr
MW	int	MPAM_WIDTH	as on axil5_slave_wr
EW	int	MECID_WIDTH	as on axil5_slave_wr
NW	int	NSAID_WIDTH	as on axil5_slave_wr
PW	int	$(DW / 64) > 0 ?$ $(DW / 64) : 1$	as on axil5_slave_wr
AWSize	int	AW + 3 +	as on axil5_slave_wr

Parameter	Type	Default	Description
WSize	int	DW + SW +	as on axil5_slave_wr
BSize	int	2 +	as on axil5_slave_wr

21.3 Ports

21.3.1 Monitor Control, Bus and Status

Port	Direction	Type	Description
cam_clear	input	logic	sync clear of the monitor's table (named as on the _mon wrapper: pin-compatible)
cfg_monitor_enable	input	logic	0 = monitor inert, table held clear
cfg_error_enable	input	logic	
cfg_timeout_enable	input	logic	
cfg_compl_enable	input	logic	
cfg_threshold_enable	input	logic	
cfg_timeout_cycles	input	logic [15:0]	MICROSECONDS of no progress; 0 = never
cfg_freq_sel	input	logic [3:0]	counter_freq_invariant LUT index
cfg_axi_pkt_mask	input	logic [15:0]	drop mask by packet type
i_mon_time	input	monitor_common_pkg::monbus_timestamp_t	
monbus_valid	output	logic	
monbus_ready	input	logic	

Port	Direction	Type	Description
monbus_packet	output	monitor_common_pkg::monitor_packet_t	
monbus_timestamp	output	monitor_common_pkg::monbus_timestamp_t	
active_transactions	output	logic [7:0]	entries in the table
error_count	output	logic [15:0]	error + timeout packets emitted
transaction_count	output	logic [31:0]	completion packets emitted
dropped_count	output	logic [15:0]	events lost to monbus backpressure since the last report
refused_count	output	logic [15:0]	commands that found no free table entry

21.3.2 Core Ports (as on axil5_slave_wr)

Port	Direction	Type	Description
acclk	input	logic	
aresetn	input	logic	
s_axil_awaddr	input	logic [AW-1:0]	
s_axil_awprot	input	logic [2:0]	
s_axil_awlock	input	logic	
s_axil_awuser	input	logic [UW-1:0]	
s_axil_awtrace	input	logic	
s_axil_awloop	input	logic [LW-1:0]	
s_axil_awmpam	input	logic [MW-1:0]	

Port	Direction	Type	Description
s_axil_awmeci d	input	logic [EW- 1:0]	
s_axil_awnsai d	input	logic [NW- 1:0]	
s_axil_awvali d	input	logic	
s_axil_awread y	output	logic	
fub_axil_awad dr	output	logic [AW- 1:0]	
fub_axil_awpr ot	output	logic [2:0]	
fub_axil_awlo ck	output	logic	
fub_axil_awus er	output	logic [UW- 1:0]	
fub_axil_awtr ace	output	logic	
fub_axil_awlo op	output	logic [LW- 1:0]	
fub_axil_awmp am	output	logic [MW- 1:0]	
fub_axil_awme cid	output	logic [EW- 1:0]	
fub_axil_awns aid	output	logic [NW- 1:0]	
fub_axil_awva lid	output	logic	
fub_axil_awre ady	input	logic	
s_axil_wdata	input	logic [DW- 1:0]	
s_axil_wstrb	input	logic [SW- 1:0]	
s_axil_wuser	input	logic [UW- 1:0]	
s_axil_wpoiso n	input	logic [PW- 1:0]	
s_axil_wvalid	input	logic	
s_axil_wready	output	logic	

Port	Direction	Type	Description
fub_axil_wdata	output	logic [DW-1:0]	
fub_axil_wstrb	output	logic [SW-1:0]	
fub_axil_wuser	output	logic [UW-1:0]	
fub_axil_wposition	output	logic [PW-1:0]	
fub_axil_wvalid	output	logic	
fub_axil_wready	input	logic	
fub_axil_bresp	input	logic [1:0]	
fub_axil_buser	input	logic [UW-1:0]	
fub_axil_btrace	input	logic	
fub_axil_bloop	input	logic [LW-1:0]	
fub_axil_bvalid	input	logic	
fub_axil_bready	output	logic	
s_axil_bresp	output	logic [1:0]	
s_axil_buser	output	logic [UW-1:0]	
s_axil_btrace	output	logic	
s_axil_bloop	output	logic [LW-1:0]	
s_axil_bvalid	output	logic	
s_axil_bready	input	logic	
busy	output	logic	

21.4 Functional Description

The core is instantiated untouched; unlike `axil5_slave_wr_mon` there is no gating of the command handshake, because the lite has no admission stall. The monitor taps the `s_`-side

command, data and response channels through three valids gated by `cfg_monitor_enable`, which also holds the table clear while low. `cfg_timeout_cycles` is a microsecond count of no progress on an entry, passed at full width; 0 means never. The active-count threshold is the parameter `ACTIVE_TRANS_THRESHOLD`, as on the `_mon` wrapper.

The lite's own behaviour – per-ID linked-list attribution, stamps instead of counters, the registered event stage, the 4-entry output queue and drop-and-count – is on the [axi_monitor_lite](#) page.

21.5 Timing Characteristics

The datapath timing is `axil5_slave_wr`'s. The monitor adds no gating to any handshake and no path into the datapath; its own paths are two short cycles (attribution, then packet formatting) and met 10 ns on an Artix-7 100T -1 with margin inside `bridge_1x2_rd_lite_mon`.

21.6 Usage Examples

```
axil5_slave_wr_monlite #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .UNIT_ID              (8'h02),
    .AGENT_ID              (16'h0014),
    .MAX_TRANSACTIONS      (8)
) u_axil5_slave_wr_monlite (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    // ... the axil5_slave_wr ports, by name ...
    .cam_clear             (1'b0),
    .cfg_monitor_enable    (1'b1),
    .cfg_error_enable      (1'b1),
    .cfg_timeout_enable    (1'b1),
    .cfg_compl_enable      (1'b0),
    .cfg_threshold_enable  (1'b0),
    .cfg_timeout_cycles    (16'd1000),
    .cfg_freq_sel          (4'd0),
    .cfg_axi_pkt_mask      (16'h0000),
    .i_mon_time            (mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .active_transactions   (),
    .error_count           (),
```

```
.transaction_count    (),  
.dropped_count        (dropped_count),  
.refused_count        (refused_count)  
);
```

In a generated bridge, `mon_preset = "lite"` selects this wrapper for every monitored port.

21.7 Design Notes

- `cam_clear` keeps the `_mon` wrapper's name so the two are pin-compatible on the monitor side; the lite has a table, not a CAM.
 - `MAX_TRANSACTIONS` defaults to 8 here against 16 on `axil5_slave_wr_mon`: size it to the port's real outstanding depth, since refused commands surface as orphans.
 - Consumers that relied on `block_ready` to bound the table get drop-and-count instead; the count is always reported, the identity of the lost events is not.
-

21.8 Related Modules

- `axil5_slave_wr_mon` – the full-monitor sibling
 - `axil5_slave_wr` – the core this wraps
 - [axi_monitor_lite](#) – the monitor itself
-

21.9 Testing

`val/amba/monitor-lite/test_axil5_slave_wr_monlite.py` exercises this module: the same TB and scenarios as `val/amba/test_axil5_slave_wr_mon.py`, with the DUT swapped.

```
source env_python  
make -C val/amba/monitor-lite run-axil5_slave_wr_monlite-gate
```

Last Updated: 2026-09-26

21.10 Navigation

- [← Back to Monitor Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)